



Escola Tècnica Superior d'Enginyeria La Salle

Treball Final de Grau

Grau en Enginyeria Multimèdia

**Desenvolupament d'un motor de jocs en C
emprant Vulkan com a API de gràfics**

Alumne
Oleguer Almuni i Orra

Professor Ponent
Ferran Ruiz Sala

**ACTA DE L'EXAMEN
DEL TREBALL FI DE CARRERA**

Reunit el Tribunal qualificador en el dia de la data, l'alumne

D.

va exposar el seu Treball de Fi de Carrera, el qual va tractar sobre el tema següent:

Acabada l'exposició i contestades per part de l'alumne les objeccions formulades pels Srs. membres del tribunal, aquest valorà l'esmentat Treball amb la qualificació de

Barcelona,

VOCAL DEL TRIBUNAL

VOCAL DEL TRIBUNAL

PRESIDENT DEL TRIBUNAL

Resum

El desenvolupament de videojocs es fa avui majoritàriament sobre motors comercials, que es prenen com una eina donada, de manera que el seu funcionament intern queda fora del que s'estudia durant la formació. Aquest treball dissenya, implementa i avalua un motor de jocs escrit en C amb un renderitzador basat en Vulkan, documentant cada decisió estructural contra les fonts primàries que la fonamenten.

El resultat és un motor funcional sobre Windows i Linux, amb capa d'abstracció de plataforma, gestió explícita de memòria, càrrega i il·luminació de recursos des de disc i una capa d'instrumentació pròpia. L'avaluació empírica mostra que el patró de transferència de dades que la bibliografia presenta com a correcte resulta prescindible quan el dispositiu exposa memòria visible des de l'amfitrió: evitar-lo redueix el temps de càrrega de geometria en un factor de 136 i de 282 sobre els dos equips provats. La conclusió principal és que la forma d'un motor modern està determinada en bona mesura per l'API sobre la qual es construeix.

Resumen

El desarrollo de videojuegos se hace hoy mayoritariamente sobre motores comerciales, que se toman como una herramienta dada, de modo que su funcionamiento interno queda fuera de lo que se estudia durante la formación. Este trabajo diseña, implementa y evalúa un motor de juegos escrito en C con un renderizador basado en Vulkan, documentando cada decisión estructural frente a las fuentes primarias que la fundamentan.

El resultado es un motor funcional sobre Windows y Linux, con capa de abstracción de plataforma, gestión explícita de memoria, carga e iluminación de recursos desde disco y una capa de instrumentación propia. La evaluación empírica muestra que el patrón de transferencia de datos que la bibliografía presenta como correcto resulta prescindible cuando el dispositivo expone memoria visible desde el anfitrión: evitarlo reduce el tiempo de carga de geometría en un factor de 136 y de 282 en los dos equipos probados. La conclusión principal es que la forma de un motor moderno está determinada en buena medida por la API sobre la que se construye.

Abstract

Game development today is carried out mostly on commercial engines, which are taken as a given tool, so that their internal workings fall outside what is studied during a degree. This work designs, implements and evaluates a game engine written in C with a Vulkan-based renderer, documenting every structural decision against the primary sources that support it.

The result is a working engine on Windows and Linux, with a platform abstraction layer, explicit memory management, loading and lighting of assets from disk, and its own instrumentation layer. The empirical evaluation shows that the data transfer pattern the literature presents as correct is dispensable when the device exposes memory visible from the host: avoiding it reduces geometry upload time by a factor of 136 and 282 on the two machines tested. The main conclusion is that the shape of a modern engine is largely determined by the API it is built upon.

Agraïments

A la meva germana, a la meva mare i al meu pare.

Aquest treball s'ha fet en gran part fora de l'horari laboral, en caps de setmana i durant un estiu sencer. Que això fos possible no depenia només de mi, i tots tres ho van fer fàcil sense haver-ho de dir.

Índex

Acrònims	12
1 Introducció	1
1.1 Context i motivació	1
1.1.1 El motor com a caixa negra	1
1.1.2 L'instrument i l'obra	1
1.1.3 Relació amb la formació del grau	1
1.1.4 El repte tècnic	2
1.1.5 Orientació professional	2
1.2 Objectius	2
1.3 Abast	3
1.4 Organització del document	3
2 Estat de l'art i marc tecnològic	5
2.1 Arquitectura d'un motor de jocs	5
2.1.1 Les capes d'un motor 3D típic	5
2.1.2 Correspondència amb el motor desenvolupat	6
2.2 De OpenGL a Vulkan: per què existeix Vulkan	7
2.2.1 El model clàssic i el seu cost	7
2.2.2 Un primer intent des de dins: AZDO	7
2.2.3 Mantle: traslladar la responsabilitat a l'aplicació	8
2.2.4 De Mantle a Vulkan	8
2.2.5 El principi de control explícit	9
2.2.6 Conseqüència: l'API imposa l'arquitectura	10
2.3 Motors existents i per què construir-ne un	10
2.3.1 El llinatge com a norma en el desenvolupament de motors	10
2.4 Projectes i materials de referència	11
3 Metodologia i planificació	12
3.1 Metodologia de desenvolupament	12
3.1.1 Fases del desenvolupament	12
3.1.2 Verificació	13
3.1.3 Documentació com a part del mètode	13
3.2 Procedència de les fonts i ús de projectes de referència	13
3.2.1 Base emprada	13
3.2.2 Aportacions pròpies	14
3.2.3 Delimitació	14
3.3 Planificació temporal	14
3.3.1 Premissa de disponibilitat	14
3.3.2 Pressupost d'hores	15
3.3.3 Distribució temporal	15
3.3.4 Desviacions	16
3.4 Eines i entorn de desenvolupament	16
3.4.1 Llenguatge, compilació i construcció	16
3.4.2 API gràfica i eines associades	16
3.4.3 Control de versions i documentació	17
3.4.4 Ús d'eines d'intel·ligència artificial	17
3.4.5 Maquinari de desenvolupament	17

4	Decisions de disseny	19
4.1	Vulkan com a API gràfica	19
4.1.1	Problema i alternatives	19
4.1.2	Criteri i decisió	19
4.1.3	Cost de la decisió	20
4.2	C com a llenguatge	20
4.2.1	Problema i alternatives	20
4.2.2	Una precisió sobre el disseny orientat a dades	21
4.2.3	Criteri i decisió	21
4.2.4	Cost de la decisió	22
4.3	El motor com a biblioteca compartida i <i>framework</i>	22
4.3.1	Problema i alternatives	22
4.3.2	Criteri i decisió	22
4.3.3	Cost de la decisió	22
4.4	Gestió explícita de memòria	23
4.4.1	Problema i alternatives	23
4.4.2	La convergència de dues exigències	23
4.4.3	Criteri i decisió	24
4.4.4	Cost de la decisió	24
4.5	Separació entre la part independent de l'API i la implementació	24
4.5.1	Problema i alternatives	24
4.5.2	Criteri i decisió	25
4.5.3	Cost de la decisió	26
4.6	Model de recursos: referències, identificadors i generacions	26
4.6.1	Problema i alternatives	26
4.6.2	Criteri i decisió	26
4.6.3	Cost de la decisió	27
5	Disseny i implementació	28
5.1	Arquitectura general i cicle de vida	28
5.1.1	Estructura	28
5.1.2	Seqüència d'arrencada	28
5.1.3	Cicle d'un <i>frame</i>	28
5.2	Gestió de memòria	29
5.2.1	Reserva etiquetada	29
5.2.2	L'assignador lineal	29
5.2.3	Memòria de dispositiu	30
5.2.4	Instrumentació	30
5.2.5	Mesura de temps de dispositiu	30
5.2.6	Limitacions i defectes detectats	31
5.3	Sincronització CPU/GPU i el bucle de <i>frame</i>	33
5.3.1	El problema	33
5.3.2	Les tres ordenacions	33
5.3.3	Per què els vectors tenen mides diferents	34
5.3.4	Recursos de <i>shader</i> i freqüència d'actualització	35
5.3.5	Supòsits sobre el dispositiu	36
5.3.6	Limitacions	37
5.4	Sistemes de recursos	38

5.4.1	El sistema de recursos	38
5.4.2	Els sistemes consumidors	38
5.4.3	Un actiu, de punta a punta	38
5.4.4	Unificació parcial	39
5.5	Resta de subsistemes	39
5.5.1	Evolució de l'estructura	41
6	Resultats i discussió	42
6.1	Metodologia de mesura	42
6.1.1	Instrument	42
6.1.2	Protocol	42
6.1.3	Limitacions de l'instrument	42
6.2	Assoliment dels objectius funcionals	43
6.2.1	Capa de plataforma	43
6.2.2	Renderitzador	44
6.2.3	Model de gestió de memòria	45
6.2.4	Validació sobre les dues arquitectures de memòria	46
6.2.5	Documentació de les decisions	46
6.2.6	Capa d'instrumentació	46
6.2.7	Avaluació empírica	46
6.3	Experiments	47
6.3.1	Què es vol mesurar i per què	47
6.3.2	Búfer intermedi enfront d'escriptura directa	47
6.4	Discussió	50
6.4.1	La condició no és la que sembla	50
6.4.2	Què queda establert i què no	50
6.4.3	L'exemple de la finestra acotada	51
6.4.4	La coherència amb el marc del capítol 2	51
6.4.5	L'asimetria entre amfitrió i dispositiu	51
6.5	Limitacions de l'avaluació	52
7	Conclusions	53
7.1	Conclusions del treball realitzat	53
7.1.1	L'API imposa l'arquitectura	53
7.1.2	Fonts independents que convergeixen	53
7.1.3	El cost de l'explicitud i on surt a compte	54
7.1.4	L'explicitud desplaça la portabilitat a l'aplicació	56
7.1.5	Una implementació de referència transmet també el seu entorn	56
7.1.6	Sobre construir l'instrument	57
7.2	Punts forts i punts febles	57
7.2.1	Punts forts	57
7.2.2	Punts febles	57
7.3	Limitacions	58
7.4	Línies de continuació	58
	Referències	60
	Annexos	62
	Annex A. Correspondència entre decisions i fonts	62
	Annex B. Compilació i execució	63
	Requisits comuns	63

Linux	64
Windows	64
Notes d'ús	64
Annex C. Registres de les mesures	64

Índex de figures

Figura 1	Model en capes d'un motor de jocs, adaptat de [1], amb indicació de les capes cobertes per Hefest.	6
Figura 2	Distribució temporal de les fases. Els períodes d'octubre i novembre de 2025 i de juliol i agost de 2026 no registren activitat, cosa coherent amb la premissa de disponibilitat exposada més amunt.	16
Figura 3	Seqüència d'una iteració del bucle principal.	29
Figura 4	Camí de càrrega d'un material, des del fitxer fins al dibuix.	39
Figura 5	El motor sobre Windows i GPU dedicada, amb la mateixa escena que s'analitza tot seguit: el cub texturat i il·luminat i el quadrilàter d'interfície. El resultat és equivalent en tots dos entorns tot i que el controlador, el sistema de finestres i el repartiment de famílies de cues del dispositiu són diferents.	44
Figura 6	El motor en execució. El cub el genera el sistema de geometria amb una normal per cara; la textura i el color difús provenen d'un fitxer de material carregat des de disc. Les tres cares visibles reben il·luminacions diferents segons l'angle que formen amb la llum direccional: la dreta hi està encarada, la superior hi queda obliqua i l'esquerra n'està girada i només rep el terme ambient. El quadrilàter del cantó superior esquerre es dibuixa a la passada d'interfície, amb un material i una projecció diferents dels del món.	45

Índex de taules

Taula 1	Correspondència entre les capes del model [1] i l'estat del motor desenvolupat.	6
Taula 2	Correspondència entre els conceptes de Mantle i els de Vulkan.	8
Taula 3	Fases del desenvolupament del motor.	12
Taula 4	Distribució estimada de la dedicació per tasca.	15
Taula 5	Entorns de desenvolupament i de mesura.	17
Taula 6	Ordre d'inicialització dels subsistemes i dependència que el motiva.	28
Taula 7	Objectes de sincronització i criteri de dimensionament.	33
Taula 8	Recursos del <i>shader</i> de materials, agrupats per freqüència d'actualització.	35
Taula 9	Inventari de subsistemes del motor.	40
Taula 10	Balanç dels objectius específics.	43
Taula 11	Temps de càrrega de recursos, en mil·lisegons, segons l'estratègia de transferència i l'equip.	48
Taula 12	Distribució del codi propi del motor per àrea, en línies no buides dels 84 fitxers font. No s'hi comptabilitzen les 7.016 línies del descodificador d'imatges de tercers que el motor incorpora.	54
Taula 13	Correspondència entre els mecanismes del motor, la seva justificació a la memòria, la font que els fonamenta i el codi que els implementa.	62

Índex de fragments de codi

Codi 1	Patró d'inicialització en dues crides. La primera invocació informa de la mida de l'estat; l'aplicació reserva aquesta quantitat de l'assignador lineal i la lliura a la segona. El subsistema no decideix, doncs, ni on resideix el seu estat ni quan s'allibera.	24
Codi 2	La frontera entre la part independent de l'API i la seva implementació. Cap paràmetre és un objecte de Vulkan: les operacions parlen de <i>frames</i> , de geometries i de materials.	25
Codi 3	Declaració de les dues reserves anteriors al comptador. La funció no reserva memòria: només n'anota una que ja s'ha produït, de manera que l'informe cobreix tot el que el motor ha demanat al sistema operatiu.	32
Codi 4	Sincronització a l'inici del <i>frame</i> . Els vectors indexats per <i>frame</i> lògic i els indexats per imatge de la cadena d'intercanvi es mantenen separats perquè les seves mides no tenen cap raó de coincidir.	34
Codi 5	Fitxer de definició d'un material. El material anomena la seva textura en lloc de contenir-la, cosa que permet que diversos materials en comparteixin una sola còpia a la memòria del dispositiu.	39
Codi 6	Construcció i execució sobre Linux.	64
Codi 7	Construcció i execució sobre Windows.	64

Acrònims

Els acrònims que provenen de paraules en una llengua diferent a la del document s'hi desplegen en cursiva. Un cop desplegats aquí, no es tornen a desplegar al llarg del text.

API *Application Programming Interface*

AZDO *Approaching Zero Driver Overhead*

CPU *Central Processing Unit*

ECTS *European Credit Transfer and Accumulation System*

GLSL *OpenGL Shading Language*

GPU *Graphics Processing Unit*

SDK *Software Development Kit*

SPIR-V *Standard Portable Intermediate Representation*

TFG *Treball de Final de Grau*

X11 *X Window System, versió 11*

XCB *X protocol C-language Binding*

1 Introducció

1.1 Context i motivació

1.1.1 El motor com a caixa negra

El desenvolupament de videojocs es fa avui, de manera aclaparadorament majoritària, sobre motors comercials. Aquesta és també la via per la qual s'hi accedeix durant la formació: el motor es pren com una eina donada, es programa al damunt i s'obtenen resultats sense necessitat de saber què hi passa a sota.

Aquesta manera de treballar és perfectament raonable —és, de fet, la decisió correcta per a gairebé qualsevol projecte que vulgui produir un joc— però deixa una part considerable del domini sense explorar. Qui hi treballa sap què fa el motor, però no com ho fa: com arriben les dades d'un actiu des del disc fins a la memòria de la targeta gràfica, com se sincronitzen el processador i el dispositiu, per què el bucle principal té les fases que té, o què determina l'ordre en què els seus subsistemes s'han d'arrencar.

La motivació d'aquest treball neix precisament d'aquesta distància: de l'interès per entendre com funciona un joc a la base, per sota de l'eina.

1.1.2 L'instrument i l'obra

Hi ha, a més, una consideració sobre el mitjà que va més enllà del pragmatisme i que convé explicitar, tot i tractar-se d'una valoració i no d'un fet.

En gairebé cap altra disciplina creativa la persona que concep i desenvolupa l'obra està capacitada per construir també l'instrument amb què la fa. El músic toca la guitarra però rarament la fabrica; el cineasta grava amb la càmera però no la dissenya. Construir l'instrument i fer-ne obra són oficis diferents, amb formacions diferents, i la separació és tan natural que amb prou feines es percep com a tal.

El videojoc constitueix una excepció poc habitual. Com que l'instrument és programari i una part considerable de qui crea videojocs ja programa, la frontera entre construir l'eina i emprar-la és molt més permeable que en qualsevol dels casos anteriors. La possibilitat existeix i, tanmateix, la indústria opta de manera aclaparadora per no exercir-la.

Aquesta elecció té raons sòlides, ja apuntades: per a la majoria de projectes, i molt especialment per a equips reduïts, construir un motor propi és una inversió que no es recupera. Però que la decisió sigui raonable no la fa inevitable. Un equip que disposa del seu propi motor pot adaptar l'eina a l'obra en lloc d'adaptar l'obra a l'eina, i és raonable pensar que això té efectes sobre el resultat quan el projecte té ambicions que l'eina genèrica no contempla.

Aquest treball no pretén demostrar aquesta tesi, que excediria de molt el seu abast. L'esmenta perquè forma part de la motivació: entendre com es construeix un motor és el requisit previ de qualsevol decisió sobre si val la pena construir-ne un.

1.1.3 Relació amb la formació del grau

El grau en Enginyeria Multimèdia proporciona els fonaments sobre els quals aquest treball es construeix. S'hi han tractat models d'il·luminació, s'han implementat mecàniques de joc i s'ha treballat amb motors, tant comercials com de complexitat reduïda, i s'hi han adquirit els coneixements de programació, matemàtiques i representació gràfica que el projecte requereix.

El que la formació no aborda —i difícilment podria abordar, per l'extensió que implica— és la construcció del motor mateix. Els models d'il·luminació s'han estudiat i aplicat, però sobre un renderitzador ja existent; les mecàniques de joc s'han implementat, però sobre un bucle principal proporcionat per l'eina. El salt entre fer servir aquests sistemes i construir-los és considerable, i és exactament el que aquest treball es proposa.

En aquest sentit, el projecte no introdueix una àrea aliena al grau sinó que aprofundeix en una de les seves: pren conceptes que s'hi han tractat en la seva vessant d'ús i els reconstrueix des dels seus fonaments, la qual cosa n'obliga a entendre les decisions que normalment queden ocultes rere l'eina.

1.1.4 El repte tècnic

A la motivació anterior s'hi afegeix l'atractiu del problema en si mateix. Un motor gràfic és una aplicació complexa: exigeix gestionar memòria de manera explícita, coordinar dos processadors que treballen de manera asíncrona, dissenyar abstraccions que sobrevisquin al creixement del sistema i mantenir un pressupost de temps estricte a cada *frame*. Poques aplicacions reuneixen aquesta combinació d'exigències.

L'elecció de Vulkan accentua aquest caràcter. Com s'argumenta al capítol 2, es tracta d'una API que trasllada deliberadament a l'aplicació responsabilitats que les generacions anteriors resolien de manera transparent. Això n'incrementa notablement la dificultat, però és també el que la converteix en un objecte d'estudi adequat: allò que altres API amaguen, aquesta obliga a decidir-ho explícitament, i cada decisió explícita és una decisió que es pot documentar i justificar.

1.1.5 Orientació professional

Finalment, el projecte respon a un objectiu professional. La indústria del videojoc és l'àmbit en què es pretén desenvolupar la trajectòria posterior al grau, i el coneixement del funcionament intern d'un motor hi té valor encara que la feina es faci sobre eines comercials: permet entendre per què aquestes eines són com són, on són els seus límits i què costa realment cada operació.

Un projecte d'aquestes característiques complementa, per tant, la resta de coneixements adquirits durant la formació, i aporta al perfil una dimensió —programació de sistemes i de gràfics a baix nivell— que el treball sobre motors comercials no desenvolupa per si mateix.

1.2 Objectius

L'objectiu general d'aquest treball és dissenyar, implementar i avaluar un motor de jocs escrit en C amb un renderitzador basat en Vulkan, documentant-ne les decisions de disseny contra les fonts primàries que les fonamenten.

Aquest objectiu general es concreta en els objectius específics següents, cadascun dels quals té una resposta explícita al capítol de resultats:

1. Implementar una capa d'abstracció de plataforma que permeti executar el motor sobre Windows i Linux sense modificacions al codi dels subsistemes superiors.
2. Implementar un renderitzador sobre Vulkan capaç de carregar geometria, textures i materials des de disc, il·luminar-los i dibuixar-los amb una càmera controlable.
3. Dissenyar un model de gestió de memòria explícit, en què la ubicació i el cicle de vida de l'estat de cada subsistema siguin decisions del motor i no del sistema d'assignació subjacent.
4. Validar el funcionament del motor sobre arquitectures de memòria unificada i de memòria dedicada, i contrastar-ne el comportament.

5. Documentar cada decisió estructural del motor amb la font primària que la fonamenta, de manera que en sigui possible la justificació independent.
6. Dotar el motor d'una capa d'instrumentació pròpia que permeti mesurar-ne el comportament sense recórrer a eines externes.
7. Avaluar empíricament un conjunt de decisions del renderitzador i quantificar l'impacte de cadascuna.

1.3 Abast

El treball comprèn el desenvolupament de les capes fonamentals d'un motor de jocs i d'un renderitzador funcional, tal com es delimita a la Taula 1: capa de plataforma, sistemes bàsics, gestió de recursos, renderitzador i tractament de dispositius d'entrada. Inclou igualment la documentació tècnica del disseny i l'avaluació empírica del comportament del renderitzador.

Queden explícitament fora de l'abast els subsistemes següents, que un motor comercial inclouria però que no són requisit per als objectius plantejats:

Simulació física i detecció de col·lisions Constitueixen per si soles un domini d'una extensió comparable a la del renderitzador.

Àudio, animació i xarxa Són subsistemes funcionals que es recolzen sobre les capes fonamentals però que no en condicionen el disseny.

Sistema de tasques concurrents El motor és d'un sol fil d'execució. Es tracta de la limitació més rellevant del treball i se'n discuteixen les implicacions al capítol 7.

Eines d'edició i canonada d'actius No hi ha editor de nivells ni procés automatitzat de conversió d'actius més enllà de la compilació dels *shaders*.

Lògica de joc L'aplicació que acompanya el motor té l'única finalitat de demostrar-ne i exercitar-ne les capacitats; no constitueix un joc.

Aquesta delimitació respon al criteri exposat a la secció sobre motors existents: l'objectiu del treball és la comprensió de les capes que qualsevol motor necessita abans de poder dibuixar res, no l'assoliment de la paritat funcional amb un producte comercial.

1.4 Organització del document

El document segueix l'ordre en què es va desenvolupar el treball: primer el marc que el fonamenta, després el mètode, les decisions, la implementació i finalment l'avaluació.

El capítol 2 estableix el marc conceptual. Presenta l'estructura en capes d'un motor de jocs i, sobretot, explica per què existeix Vulkan: quins límits del model gràfic anterior el van motivar, quin intent es va fer de resoldre'ls sense trencar-lo i quin principi governa l'API resultant. Aquesta darrera part no és context decoratiu, sinó la premissa que sosté els dos capítols següents, ja que la manera com l'API reparteix responsabilitats determina quins subsistemes ha de tenir qualsevol motor construït al damunt. El capítol es tanca situant el treball respecte dels motors existents i dels projectes que li han servit de referència.

El capítol 3 descriu el mètode. Detalla com s'ha organitzat el desenvolupament, com s'ha verificat i quin paper hi ha tingut la documentació tècnica. Inclou una secció dedicada a la procedència de les fonts, que delimita amb precisió què s'ha pres d'una implementació de referència i què constitueix aportació pròpia. Es completa amb la planificació temporal i amb la descripció de l'entorn de desenvolupament i de mesura.

El capítol 4 recull les sis decisions estructurals del motor: l'API gràfica, el llenguatge, el repartiment del control del flux, la gestió de memòria, la frontera entre la part independent de l'API i la seva implementació, i el model de recursos. Cadascuna s'exposa amb la mateixa estructura —problema, alternatives, criteri i cost— i es fonamenta en la font corresponent.

El capítol 5 descriu com s'han implementat aquestes decisions. Tracta en profunditat l'arquitectura general i el cicle d'un *frame*, la gestió de memòria i la instrumentació, la sincronització entre processador i dispositiu, i els sistemes de recursos; la resta de subsistemes s'hi recullen en un inventari. La secció dedicada a la sincronització inclou els supòsits sobre el maquinari que va caldre corregir perquè el motor s'executés sobre l'entorn d'aquest treball.

El capítol 6 presenta l'avaluació. Comença explicitant què mesura la instrumentació i quines limitacions té, atès que d'això depèn quines afirmacions poden sostenir-se. Segueix amb el balanç dels objectius, l'experiment sobre estratègies de transferència de dades cap a la memòria del dispositiu, la discussió dels resultats i la delimitació del seu abast.

El capítol 7 recull les conclusions, els punts forts i febles del treball, les limitacions del motor resultant i les línies de continuació, ordenades segons la relació entre el que aporten i el que costen.

Els annexos contenen el material de consulta: la correspondència detallada entre les decisions de disseny, les fonts que les fonamenten i el codi que les implementa; les instruccions de compilació i execució sobre les dues plataformes; i els registres crus de les mesures del capítol 6.

2 Estat de l'art i marc tecnològic

2.1 Arquitectura d'un motor de jocs

Un motor de jocs no és un programa monolític, sinó un conjunt de subsistemes organitzats en capes, on les capes superiors depenen de les inferiors però no a la inversa [1]. Quan una capa inferior depèn d'una de superior es produeix una dependència circular, situació indesitjable en qualsevol sistema de programari perquè genera acoblament entre subsistemes, en dificulta la verificació i n'impedeix la reutilització.

Aquest principi no és una consideració merament teòrica. En un motor com el que es descriu en aquest treball, determina l'ordre en què els subsistemes s'han d'inicialitzar i, per tant, condiciona directament el disseny de la seqüència d'arrencada que es tracta als capítols 4 i 5.

2.1.1 Les capes d'un motor 3D típic

Un motor 3D complet es pot descompondre en setze capes [1], que al seu torn s'agrupen en quatre blocs segons la seva naturalesa.

El primer bloc no forma part del motor, sinó del seu fonament: el maquinari objectiu, els controladors de dispositiu, el sistema operatiu i els SDK de tercers i programari intermediari. Determinen les possibilitats i els límits de tot el que hi ha per sobre, però no els escriu qui desenvolupa el motor.

El segon bloc és el fonament del motor pròpiament dit. La capa d'independència de plataforma encapsula les diferències entre sistemes operatius i biblioteques natives, de manera que la resta del motor pugui ignorar-les. Els sistemes bàsics apleguen les utilitats transversals que tota aplicació gran necessita: assercions, gestió de memòria, biblioteca matemàtica, estructures de dades i registre d'esdeveniments. El gestor de recursos ofereix una interfície unificada per accedir als actius del joc, encara que cada motor el resol de manera diferent, des d'esquemes centralitzats fins a solucions *ad hoc* on el programador accedeix directament als fitxers [1].

El tercer bloc conté els subsistemes funcionals: el renderitzador, les eines de perfilatge i depuració, la detecció de col·lisions i la física, l'animació, els dispositius d'interfície humana, l'àudio i la xarxa i el multijugador en línia. Són els que donen al motor les seves capacitats visibles, i cadascun és, en si mateix, un sistema complex.

El quart bloc és el més proper al joc concret: els sistemes fonamentals de joc i els subsistemes específics del joc. És on la frontera entre motor i joc es difumina, i on cada projecte pren decisions pròpies.

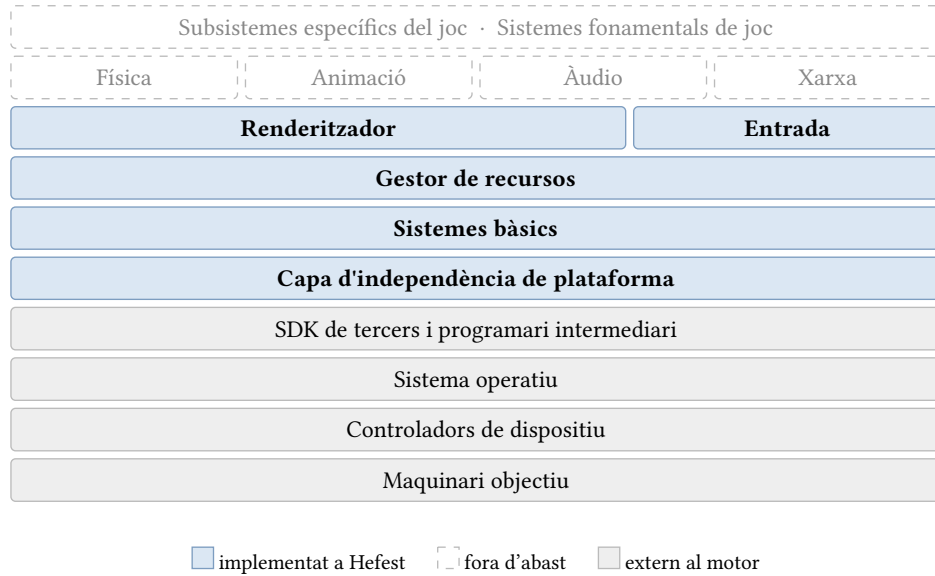


Figura 1: Model en capes d'un motor de jocs, adaptat de [1], amb indicació de les capes cobertes per Hefest.

2.1.2 Correspondència amb el motor desenvolupat

La Figura 1 situa el treball dins d'aquest model. Hefest implementa les tres capes fonamentals — independència de plataforma, sistemes bàsics i gestor de recursos— i dues de les funcionals: el renderitzador i els dispositius d'interfície humana. La resta queda explícitament fora de l'abast, tal com s'estableix a la secció corresponent del capítol 1.

Aquesta delimitació no és arbitrària. Les capes implementades són precisament les que qual-
sevol motor necessita abans de poder mostrar res per pantalla, i les que constitueixen els sistemes de suport del motor [1]. Les capes omeses —física, animació, àudio, xarxa— són subsistemes funcionals que es recolzen sobre les anteriors però que no en són requisit previ. Un motor sense àudio continua sent un motor; un motor sense gestió de memòria ni capa de plataforma, no.

Taula 1: Correspondència entre les capes del model [1] i l'estat del motor desenvolupat.

Capa	Estat a Hefest	On es tracta
Independència de plataforma	Implementada (Win32, XCB/X11)	Cap. 4 i 5
Sistemes bàsics	Implementats	Cap. 5
Gestor de recursos	Parcial: textures i materials	Cap. 4 i 5
Renderitzador	Implementat sobre Vulkan	Cap. 4, 5 i 6
Perfilatge i depuració	Parcial: instrumentació pròpia	Cap. 5 i 6
Dispositius d'interfície humana	Implementada: teclat i ratolí	Cap. 5
Col·lisions i física	No implementada	Fora d'abast
Animació	No implementada	Fora d'abast
Àudio	No implementada	Fora d'abast
Xarxa i multijugador	No implementada	Fora d'abast
Sistemes fonamentals de joc	No implementats	Fora d'abast
Subsistemes específics del joc	Mínims: aplicació de proves	Cap. 5

Convé remarcar una asimetria que la Taula 1 no mostra: el fet que una capa estigui implementada no vol dir que ho estigui de manera completa. El gestor de recursos, per exemple, existeix en forma de dos sistemes especialitzats —textures i materials— sense la capa genèrica d'accés unificat als actius que el model preveu [1]. Aquesta mena de limitacions es documenten al lloc corresponent de cada subsistema i es recullen de manera conjunta a les conclusions.

Cal precisar el que la taula recull com a perfilatge i depuració: el motor disposa de comptadors d'ocupació de memòria per etiqueta i d'una capa d'instrumentació que mesura el cost de cada fase del *frame*, tant al processador com al dispositiu. No hi ha, en canvi, cap eina de depuració visual ni representació en pantalla d'aquestes dades, de manera que la cobertura d'aquesta capa és parcial. El capítol 6 en detalla l'abast i les limitacions.

2.2 De OpenGL a Vulkan: per què existeix Vulkan

Aquesta secció no és un excurs històric. La forma d'un motor de jocs modern està determinada en bona part per la naturalesa de l'API gràfica sobre la qual es construeix, i entendre per què Vulkan és com és equival a entendre per què el motor descrit en aquest treball ha de contenir un assignador de memòria, un model explícit de ritme de *frames* i un sistema de recursos propi.

2.2.1 El model clàssic i el seu cost

Durant més de dues dècades, la interacció entre aplicació i maquinari gràfic va seguir un model en què el controlador feia gran part de la feina difícil: la gestió dels riscos d'accés concurrent a la memòria, la sincronització entre processador i GPU, i l'assignació de memòria de vídeo [2]. Aquest repartiment de responsabilitats tenia virtuts considerables. El model era estable —codi escrit vint anys enrere continuava funcionant—, era senzill d'utilitzar, i va ser suficient per a una gamma molt àmplia d'aplicacions [2].

El problema és que aquesta comoditat té un preu mesurable. El sobrecost del controlador es tradueix en cicles de processador, ocupació de la memòria cau, consum energètic i, indirectament, rendiment de la GPU [2]. Per a aplicacions poc exigents el preu és irrellevant; per a jocs amb escenes complexes deixa de ser-ho. Els límits identificats del model clàssic són concrets: no escala amb la complexitat de l'escena, el seu model de fils d'execució és inadequat per a processadors multinucli, i l'abstracció del maquinari que proposa ja no es correspon amb l'arquitectura de les GPU modernes [2].

El diagnòstic que el mateix consorci Khronos en fa uns anys més tard és més detallat i val la pena enumerar-lo, perquè cada punt té una conseqüència directa sobre el disseny d'un motor [3]. El model de programació no es correspon amb el maquinari de la GPU —especialment en dispositius mòbils— i és el controlador qui n'amaga el desajust. El cost en processador prové sobretot de la validació d'estat i del seguiment de dependències que el controlador ha de fer. Els controladors resultants són complexos i difícils de predir, amb errors i camins ràpids diferents a cada GPU. I, de manera fonamental, el model és d'un sol fil d'execució, cosa que impedeix aprofitar els processadors multinucli.

2.2.2 Un primer intent des de dins: AZDO

La primera resposta a aquests límits no va ser substituir l'API, sinó intentar resoldre'ls dins del marc existent. Aquesta és exactament la pregunta que planteja la iniciativa AZDO —*Approaching Zero Driver Overhead*—: si els inconvenients del model clàssic es poden corregir sense renunciar-ne als avantatges [2].

La resposta tècnica consisteix a desacoblar el processador de la GPU i fer que es comuniquin a través de la memòria en lloc de fer-ho a través de crides a l'API. L'aplicació escriu a memòria

les ordres de dibuix indirectes, els objectes de *buffer* i les referències a textures; la GPU llegeix les ordres directament de memòria; i la intervenció del controlador es redueix al mínim. Com que la comunicació passa per memòria i no per l'API, diversos fils d'execució hi poden escriure simultàniament sense necessitat de cap mecanisme nou [2].

Els resultats reportats són d'entre cinc i quinze vegades el rendiment original en els casos limitats pel controlador [2]. Es tracta, doncs, d'una millora substancial obtinguda sense trencar res: AZDO destaca explícitament com a avantatges que la proposta no requereix un model d'objectes nou ni invalida les aplicacions existents [2].

Aquest darrer punt és el que fa que AZDO sigui rellevant per a entendre Vulkan. La direcció que assenyala —comunicar a través de memòria, minimitzar la intervenció del controlador, permetre el paral·lelisme real— és precisament la que Vulkan acabarà adoptant. La diferència és que Vulkan la porta fins al final, i per fer-ho pren la decisió contrària respecte de la compatibilitat: sí que defineix un model d'objectes nou i sí que trenca amb les aplicacions existents.

2.2.3 Mantle: traslladar la responsabilitat a l'aplicació

El pas següent el marca Mantle, l'API que AMD va publicar el 2014 i que documenta el 2015 [4]. El diagnòstic de partida és el mateix: els models de programació de la generació anterior no són solucions idònies en escenaris on el desenvolupador necessita un control més estret del sistema gràfic i un sobrecost d'execució menor [4].

La solució proposada s'articula al voltant d'un principi que val la pena citar literalment, perquè és el que explica la forma de tot el que ve després:

L'aplicació és l'àrbitre del renderitzat correcte i l'únic dipositari de l'estat persistent. L'anàlisi de les API actuals indica que una solució eficient per a lots petits només es pot assolir quan el controlador és tan mancat d'estat com sigui possible.

— [4], traducció pròpia

D'aquest principi se'n deriva un segon, encara més explícit: quan la implementació genèrica d'una funcionalitat s'ha demostrat massa ineficient en altres API, la responsabilitat es trasllada a l'aplicació, perquè aquesta coneix millor el context de renderitzat i pot aplicar estratègies d'optimització més intel·ligents. L'exemple que se'n dona és precisament el que més afecta l'arquitectura d'un motor: la gestió de la memòria de vídeo passa a ser responsabilitat de l'aplicació [4].

El manifest per a desenvolupadors que tanca el capítol introductori de Mantle és inusualment franc sobre què implica això. El controlador deixa de proporcionar seguretat, millores de rendiment i solucions de compromís; no crea fils d'execució addicionals d'amagat de l'aplicació, no fa validació extensiva als camins crítics de rendiment i no recompila *shaders* en segon pla. A canvi, el desenvolupador ha d'assumir la validació, la correcció d'usos indeguts de l'API i la sincronització. El document ho formula sense embuts: aquesta responsabilitat addicional és el preu que s'ha de pagar per obtenir-ne els avantatges [4].

2.2.4 De Mantle a Vulkan

La correspondència entre els conceptes de Mantle i els de Vulkan és directa i es pot comprovar comparant l'estructura dels dos documents de referència, tal com recull la Taula 2.

Taula 2: Correspondència entre els conceptes de Mantle i els de Vulkan.

Concepte a Mantle [4]	Equivalent a Vulkan [5]
<i>GPU memory heaps</i> , objectes de memòria	VkMemoryHeap, VkDeviceMemory
Cues i <i>command buffers</i>	VkQueue, VkCommandBuffer
<i>Command buffer fences</i>	VkFence
<i>Queue semaphores</i>	VkSemaphore
<i>Events</i>	VkEvent
<i>Descriptor sets</i> i les seves actualitzacions	VkDescriptorSet
Imatges i vistes d'imatge	VkImage, VkImageView
Estats de recurs i preparació	Disposicions d'imatge i barreres
Estat estàtic i estat dinàmic	Estat de <i>pipeline</i> i estat dinàmic
<i>Graphics pipelines</i>	VkPipeline
Capa de validació	Capes de validació

La coincidència no es limita a la nomenclatura. La descripció del model d'execució és pràcticament la mateixa als dos documents: l'execució dels *command buffers* dins d'una cua és sèrie, però cues diferents poden executar-se de manera asíncrona, i és responsabilitat de l'aplicació sincronitzar-les; l'enviament a una cua retorna el control a l'aplicació abans que la feina s'executi [4]. La formulació de Vulkan és equivalent: les ordres d'enviament a cua haurien de retornar tan bon punt la feina s'ha enviat, sense esperar-ne la finalització, i no hi ha restriccions d'ordenació implícites entre operacions de cues diferents [5].

El mateix passa amb el model de memòria. Tots dos documents descriuen un dispositiu que exposa diversos *heaps* amb propietats diferents, adverteixen que no tota la memòria de la GPU és necessàriament accessible des del processador, i prescriuen que l'aplicació no faci cap suposició sobre aquesta visibilitat sinó que consulti les propietats que el sistema reporta [4], [5]. Mantle ja hi descriu, el 2015, el patró que se'n deriva: quan un *heap* concret no és accessible des del processador, les dades s'hi carreguen mitjançant operacions de còpia de GPU des d'un objecte de memòria que sí que ho és [4]. Aquesta advertència no és una formalitat, i el capítol 6 en mostra les conseqüències pràctiques sobre maquinari amb memòria unificada.

Aquesta correspondència no és casual. Les discussions de disseny de Vulkan comencen l'octubre de 2012 i s'acceleren entre el juliol i l'agost de 2014 amb dos fets decisius: el compromís de diversos desenvolupadors de programari clau i la cessió de Mantle per part d'AMD [3]. L'API resultant es publica el febrer de 2016, acompanyada de controladors conformes de quatre fabricants, un compilador de GLSL a SPIR-V i eines de depuració i validació [3].

2.2.5 El principi de control explícit

El fonament de Vulkan es formula com un pacte recíproc, i no com una simple transferència de càrrega cap a l'aplicació [3]. L'aplicació es compromet a dir al controlador què farà, amb prou detall perquè no ho hagi d'endevinar i en el moment en què el controlador necessita saber-ho. A canvi, el controlador es compromet a fer exactament allò que se li ha demanat, quan se li ha demanat, i molt de pressa.

El contrast amb el model anterior és explícit. Les API clàssiques permeten especificar informació important molt tard i canviar-la en qualsevol moment, la qual cosa és còmoda però té un cost de rendiment elevat; i els seus controladors sovint ajornen feina, la traslladen a un altre

fil o fins i tot ignoren ordres basant-se en suposicions sobre la intenció de l'aplicació [3]. Els controladors de Vulkan no ho fan.

D'aquest pacte se'n deriva la política de gestió d'errors, que és igualment explícita: Vulkan està optimitzat per a aplicacions correctes i, en general, no comprova usos indeguts, no fa seguiment de dependències i no proporciona seguretat entre fils; incomplir-ne les regles produeix comportament indefinit [3]. Sí que informa dels errors que l'aplicació no pot preveure —manca de memòria, pèrdua del dispositiu— i deixa la resta de la comprovació a les capes de validació, pensades per activar-se durant el desenvolupament. Aquesta divisió és la que justifica que el motor descrit en aquest treball habiliti la validació únicament a les compilacions de depuració, tal com es detalla al capítol 4.

2.2.6 Conseqüència: l'API imposa l'arquitectura

Convé tornar a la llista de responsabilitats que el model clàssic assignava al controlador: gestió de riscos d'accés, sincronització i assignació de memòria [2]. Vulkan les trasllada totes tres a l'aplicació. Aquest trasllat no és un detall d'implementació, sinó el fet que determina quins subsistemes ha de tenir qualsevol motor construït sobre aquesta API:

Assignació de memòria si el controlador no gestiona la memòria del dispositiu, el motor necessita una estratègia pròpia d'assignació i subassignació, i ha de decidir com transfereix les dades cap a memòria no visible des del processador.

Sincronització si no hi ha ordenació implícita entre operacions, el motor ha de construir explícitament el seu model de ritme de *frames* amb primitives de sincronització, i ha de decidir quantes imatges processa simultàniament.

Estat i validació si el controlador és mancat d'estat i no valida als camins crítics, el motor ha de decidir on col·loca l'estat de renderitzat i com n'obté diagnòstic durant el desenvolupament sense pagar-ne el cost en producció.

Aquests tres punts són, precisament, l'índex del capítol 4. Les decisions de disseny que s'hi documenten no responen a preferències estètiques sinó a obligacions que l'API imposa, i és per això que aquesta secció ocupa el lloc que ocupa dins de l'estat de l'art.

2.3 Motors existents i per què construir-ne un

El panorama actual dels motors de propòsit general està dominat per unes poques solucions madures. Unity [6] i Unreal Engine [7] ocupen la major part del mercat comercial, mentre que Godot [8] s'ha consolidat com l'alternativa de codi obert més estesa. Totes tres ofereixen editors visuals, sistemes de física, àudio, animació i interfície d'usuari, exportació a múltiples plataformes i comunitats extenses de documentació i recursos.

Davant d'aquesta oferta, construir un motor propi no té sentit com a proposta de producte. Cap projecte individual pot competir amb l'abast d'aquestes eines, i per a desenvolupar un joc la decisió racional és gairebé sempre fer servir-ne una. La justificació d'aquest treball és d'una altra naturalesa: allò que aquests motors aporten com a valor —abstreure la complexitat del maquinari gràfic, de la gestió de memòria i de la sincronització— és precisament allò que aquí constitueix l'objecte d'estudi. Un motor comercial resol aquests problemes i n'amaga la solució; l'objectiu d'aquest projecte és entendre-los.

2.3.1 El llinatge com a norma en el desenvolupament de motors

Convé situar aquesta decisió en el context de com es construeixen realment els motors de jocs, perquè la imatge del motor escrit des de zero és poc representativa de la pràctica del sector. Els

motors deriven habitualment d'altres motors. La tecnologia de Quake, per exemple, va donar lloc a una successió de títols i motors que passa per Sin, F.A.K.K. 2 i els diversos Medal of Honor, i fins i tot el motor Source de Valve en té arrels llunyanes [1].

Aquesta pràctica té una dimensió pedagògica reconeguda. El codi font de Quake i Quake II és lliurement accessible i, tot i estar escrit íntegrament en C i haver quedat desfasat, es considera un exemple net i ben estructurat de com es construeixen motors de nivell industrial; la recomanació explícita és descarregar-lo, compilar-lo i analitzar-ne el funcionament executant-lo pas a pas sota un depurador [1].

Estudiar una implementació existent i construir-hi a sobre no és, doncs, una drecera respecte del mètode habitual, sinó el mètode habitual. Aquest treball adopta aquesta aproximació de manera deliberada i explícita, i el capítol 3 en detalla la metodologia i les fonts.

2.4 Projectes i materials de referència

El desenvolupament d'aquest motor s'ha recolzat en un conjunt de projectes de codi obert i materials formatius que aborden problemes equivalents. Se'n detallen aquí els que han tingut un pes real, per ordre d'importància.

Kohi Sèrie de desenvolupament d'un motor de jocs en C i Vulkan, publicada com a llista de reproducció de vídeos [9] amb el codi font associat disponible públicament [10]. És la implementació de referència principal del projecte: n'estableix l'arquitectura general, l'ordre d'incorporació dels subsistemes i bona part de les convencions de nomenclatura. El capítol 3 en documenta l'ús concret i delimita què s'ha pres com a base i què constitueix aportació pròpia.

Handmade Hero Sèrie que desenvolupa un joc complet en C des de zero, sense biblioteques externes [11]. Rellevant sobretot pel tractament de la gestió de memòria basada en regions i per l'argumentació sobre control explícit dels recursos.

bgfx Biblioteca de renderitzat multiplataforma que abstruï diverses API gràfiques rere una interfície comuna [12]. Serveix com a referència de com es dissenya la frontera entre la part del renderitzador independent de l'API i la part específica de cada implementació, qüestió que es tracta al capítol 4.

Exemples de Vulkan de Sascha Willems Col·lecció extensa d'exemples que cobreix la major part de les funcionalitats de l'API [13]. Emprada com a referència puntual de contrast en la implementació de tècniques concretes.

A aquests projectes cal afegir-hi les dues obres que han servit de fonament teòric al llarg de tot el treball: *Game Engine Architecture* [1], per a l'arquitectura de motors, i l'especificació de Vulkan [5], com a font normativa per a tot el que afecta l'API gràfica.

3 Metodologia i planificació

3.1 Metodologia de desenvolupament

El desenvolupament s'ha organitzat de manera incremental per subsistemes. Cada subsistema s'ha portat fins a un estat funcional, s'ha integrat amb els anteriors i se n'ha documentat el disseny abans de començar el següent. El resultat és un historial d'una cinquantena de *commits* sobre el codi, repartits entre l'agost de 2025 i el setembre de 2026, en què cada un correspon a una unitat de treball coherent i identificable.

L'ordre d'aquesta incorporació no és arbitrari. L'estructura en capes descrita a la Figura 1 imposa un ordre de dependències que en determina bona part: no es pot construir el renderitzador sense una capa de plataforma que proporcioni una finestra, ni el sistema de textures sense un renderitzador capaç de crear recursos a la GPU. Dins d'aquesta restricció s'ha prioritzat sempre arribar com abans millor a un estat executable, encara que fos mínim, per disposar de verificació empírica contínua.

3.1.1 Fases del desenvolupament

Retrospectivament, el treball s'agrupa en quatre fases amb caràcter metodològic diferenciat, recollides a la Taula 3.

Taula 3: Fases del desenvolupament del motor.

Fase	Període	Contingut
Posada en marxa	ago. – set. 2025	Registre i assercions, capa de plataforma, capa d'aplicació, memòria, esdeveniments, entrada, arrencada de Vulkan fins a esborrar la pantalla, biblioteca matemàtica.
Refactorització	des. 2025	Revisió de contenidors i tipus booleans, projecte de proves unitàries, assignador lineal, adopció del patró d'inicialització en dues crides i migració de tots els subsistemes existents.
Consolidació	des. 2025 – jun. 2026	Sistema de fitxers, canonada gràfica completa, uniformes i descriptors, constants d'inserció, textures des de disc, sistema de textures, suport per a Linux, documentació i adaptació a GPU integrades.
Ampliació	set. 2026	Sistemes de materials, de geometria i de recursos amb carregadors especialitzats; suport per a múltiples passades de renderitzat i passada d'interfície d'usuari; il·luminació direccional; capa d'instrumentació.

La fase de refactorització mereix un comentari, perquè il·lustra com ha funcionat realment el procés. El patró d'inicialització en dues crides que caracteritza tots els subsistemes del motor no es va dissenyar per endavant: va aparèixer en aquesta fase, quan ja existien set subsistemes amb esquemes d'arrencada heterogenis, i es va aplicar retroactivament a tots. El mateix va passar amb l'assignador lineal, que es va introduir simultàniament perquè el patró nou el necessitava.

L'arquitectura, doncs, no estava completament planificada des de l'inici; una part es va establir només després de tenir prou subsistemes com per veure què tenien en comú.

3.1.2 Verificació

La verificació s'ha fet per tres vies complementàries, cadascuna amb un abast diferent.

La primera és l'execució de l'aplicació de proves, que ha funcionat com a comprovació contínua d'integració: qualsevol regressió en el camí de renderitzat es manifesta immediatament de manera visible.

La segona són les capes de validació de Vulkan, que constitueixen l'eina principal de correcció per a tot el codi gràfic. Com s'ha vist a la secció corresponent del capítol 2, l'API no comprova els usos indeguts als camins crítics, de manera que aquestes capes són l'únic mecanisme sistemàtic de detecció d'errors d'ús durant el desenvolupament.

La tercera són les proves unitàries, incorporades a la fase de refactorització juntament amb un projecte de proves independent. La seva cobertura és limitada —únicament l'assignador lineal i la taula de dispersió— perquè s'han aplicat només als components amb comportament aïllable i verificable sense context gràfic. Aquesta limitació es discuteix al capítol 7.

3.1.3 Documentació com a part del mètode

Paral·lelament al codi s'ha mantingut un corpus de documentació tècnica de 35 documents i unes 2.200 línies, amb un document per subsistema i una estructura fixa: propòsit, fitxers, interfície pública, funcionament, decisions de disseny i justificació, limitacions conegudes i referències creuades.

Aquesta documentació no és un subproducte redactat al final, sinó una part del mètode de treball, i respon a un objectiu concret: obligar a explicitar la justificació de cada decisió en el moment de prendre-la, i registrar-ne les limitacions mentre encara són presents. Bona part del material dels capítols 4 i 7 en prové directament. La secció de limitacions conegudes de cada document ha resultat especialment útil, perquè documentar allò que un subsistema no fa bé és el que permet, més endavant, distingir una decisió deliberada d'un descuit.

3.2 Procedència de les fonts i ús de projectes de referència

Tal com s'ha exposat a la secció sobre el llinatge en el desenvolupament de motors, construir sobre una implementació existent és la pràctica habitual del sector i una via d'aprenentatge reconeguda [1]. Aquest treball hi recorre de manera deliberada, i aquesta secció en detalla l'abast amb precisió.

3.2.1 Base emprada

La implementació de referència principal ha estat la sèrie Kohi [9], amb el codi font associat [10]. Se n'han pres l'arquitectura general del motor, l'ordre d'incorporació dels subsistemes i les convencions de nomenclatura, que en el codi d'aquest projecte apareixen amb el prefix `h` en lloc del prefix original.

La sèrie s'ha fet servir com a implementació de referència, no com a transcripció. El criteri de treball ha estat que cada decisió estructural s'estudiés de manera independent contra les fonts primàries —*Game Engine Architecture* [1] per a les qüestions d'arquitectura, i l'especificació de Vulkan [5] per a tot el que afecta l'API gràfica— abans de documentar-la. El capítol 4 recull el resultat d'aquest exercici, i l'Annex A en conté la correspondència detallada entre cada decisió i la font que la fonamenta.

3.2.2 Aportacions pròpies

Els elements següents no provenen de la implementació de referència i són verificables a l'historial del repositori.

Adaptació a maquinari no previst El *commit* a0f9a24 corregeix quatre supòsits que el codi feia sobre el dispositiu i que no es compleixen sobre una GPU integrada ni sobre un compositor que imposi una extensió de finestra pròpia. Cap dels quatre es manifesta al maquinari sobre el qual es desenvolupa la implementació de referència, de manera que localitzar-los va exigir entendre què garanteix l'API i què no, en lloc de reproduir-ne el codi. El capítol 5 en detalla els quatre casos.

Portabilitat i conformitat El *commit* 317f497 elimina els vectors de longitud variable del codi i resol diversos problemes que impediien l'execució sobre Linux. Els *commits* 0019163 i 68fdf25 corregeixen dos defectes detectats en portar el motor a l'equip amb GPU dedicada: un punter que sobrevivia a l'àmbit de l'objecte apuntat durant la creació de la cadena d'intercanvi, i un error en el pas de l'estat al subsistema de registre. Tots dos són comportament indefinit present també a l'altre equip, on no es manifestaven. S'hi afegeix un tercer defecte d'aquesta mena, detectat llegint el codi en comptes d'executar-lo: la inicialització del sistema d'esdeveniments netejava vuit bytes en lloc de la taula sencera, i només funcionava perquè l'assignador lliura la memòria ja neta.

Corpus de documentació tècnica Els 35 documents descrits a la secció anterior, amb la justificació i les limitacions de cada subsistema.

Anàlisi de correspondència amb les fonts primàries El document que relaciona cada decisió de disseny del motor amb la secció corresponent de les fonts normatives, recollit a l'Annex A.

Capa d'instrumentació El subsistema que mesura el cost de cada fase del *frame*, dissenyat a partir de la descripció de perfilatge integrat i d'estadístiques de memòria d'[1]. Manté una finestra mòbil de mostres per obtenir xifres estables i acumuladors independents del bucle per a les operacions que només succeeixen a l'arrencada. Mesura per separat el temps de l'amfitrió i el del dispositiu, aquest últim mitjançant consultes de marca de temps [5]. El capítol 5 en descriu el disseny i el capítol 6 el fa servir.

Avaluació empírica El disseny experimental i les mesures del capítol 6.

3.2.3 Delimitació

Convé ser explícit sobre el revers d'aquesta llista: la major part dels subsistemes del motor segueixen l'estructura de la implementació de referència, i el treball propi en aquests casos consisteix a haver-los comprès, integrats, depurats i documentats, no a haver-los concebut. Aquesta distinció es manté al llarg del capítol 5, on cada subsistema indica quina és la seva procedència.

3.3 Planificació temporal

3.3.1 Premissa de disponibilitat

La planificació d'aquest treball es va dissenyar sobre una restricció coneguda des de l'inici: el projecte s'ha desenvolupat de manera simultània a una activitat laboral a jornada completa, essent el TFG l'única assignatura matriculada del curs. Aquesta circumstància no permet una dedicació uniforme al llarg de l'any acadèmic, i la planificació ho va incorporar com a premissa en lloc de tractar-ho com una contingència.

L'estratègia adoptada va ser concentrar l'esforç als períodes d'alta disponibilitat i mantenir una cadència reduïda però contínua durant la resta del temps. En conseqüència, una part substancial del desenvolupament —tota la fase de posada en marxa descrita a la Taula 3— es va dur a terme durant l'estiu de 2025, i les fases posteriors es van planificar amb una càrrega setmanal compatible amb la jornada laboral.

3.3.2 Pressupost d'hores

El treball té assignats 16 crèdits ECTS. A raó de vint-i-cinc hores per crèdit, el pressupost de dedicació és de quatre-centes hores, i és sobre aquesta xifra que s'ha distribuït la càrrega de les tasques.

La Taula 4 en recull el repartiment.

Taula 4: Distribució estimada de la dedicació per tasca.

Tasca	Hores
Estudi de les fonts i de la implementació de referència	150
Escriptura de codi nou	55
Depuració i adaptació a maquinari divers	45
Redacció de la memòria	30
Documentació tècnica dels subsistemes	30
Revisió i refactorització de codi existent	25
Disseny, execució i anàlisi de l'avaluació	25
Sistema de construcció i entorn de desenvolupament	20
Seguiment i reunions de direcció	15
Total	395

Dues línies mereixen comentari perquè la seva magnitud pot sorprendre.

L'estudi de les fonts és, amb diferència, la partida més gran, i respon a la naturalesa del treball tal com s'ha plantejat al capítol 1: l'objectiu no era produir codi sinó entendre'l, i cada decisió estructural del capítol 4 va requerir contrastar la implementació de referència amb la bibliografia i amb l'especificació de l'API. Inclou també el seguiment de la sèrie de referència, activitat que combina estudi i escriptura de codi en proporcions difícils de separar.

La depuració i adaptació a maquinari divers apareix separada de l'escriptura de codi perquè és de naturalesa diferent i perquè constitueix una part substancial de l'aportació pròpia del treball. Hi entren la correcció dels supòsits sobre el dispositiu documentats a §5.3, el port a Linux, el port a Windows i els dos defectes de comportament indefinit que en van resultar.

3.3.3 Distribució temporal

La Figura 2 situa al calendari les fases i les dues activitats transversals que les acompanyen. Les barres reflecteixen els períodes amb activitat efectiva, determinats a partir de l'historial del repositori, i no una previsió teòrica: mostren, per tant, tant la feina com les dues aturades que la premissa de disponibilitat anticipava.

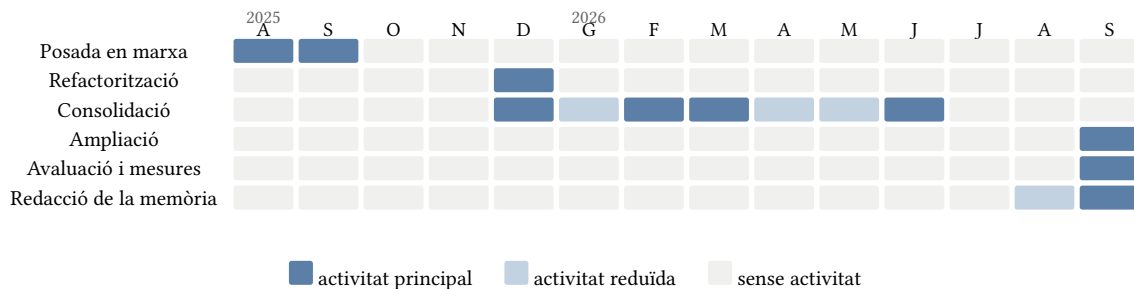


Figura 2: Distribució temporal de les fases. Els períodes d’octubre i novembre de 2025 i de juliol i agost de 2026 no registren activitat, cosa coherent amb la premissa de disponibilitat exposada més amunt.

3.3.4 Desviacions

La planificació ha resultat raonablement ajustada al pressupost, amb una dedicació estimada de 395 hores sobre les 400 previstes, però la seva distribució temporal no ha estat la que s’anticipava. Es recullen aquí les dues desviacions més rellevants.

La primera afecta el repartiment entre activitats. L’estudi de les fonts ha consumit una proporció de la dedicació superior a la prevista inicialment, en part perquè contrastar cada decisió amb la bibliografia i amb l’especificació és més lent que adoptar-la, i en part perquè aquest contrast va obligar en dues ocasions a refer conclusions ja escrites. Aquestes dues ocasions es documenten al capítol 7 i, vistes en retrospectiva, són el que distingeix aquest treball d’una reproducció de la implementació de referència; el temps addicional que van consumir es considera, doncs, ben invertit.

La segona afecta el calendari. El sistema de materials i els subsistemes que en depenen — geometria, recursos i les passades de renderitzat addicionals— es van endarrerir respecte de la previsió i es van concentrar en la fase final. L’adaptació a maquinari no previst, per la seva banda, no estava planificada en absolut: va sorgir en detectar que el motor no s’executava sobre l’equip de desenvolupament, i va consumir temps que no tenia assignació. Va acabar sent una de les aportacions pròpies del treball i la base de bona part del capítol 5, de manera que la desviació té un resultat defensable, però no deixa de ser una desviació.

3.4 Eines i entorn de desenvolupament

3.4.1 Llenguatge, compilació i construcció

El motor està escrit en C i es compila exclusivament amb Clang, en la versió 22.1.8 sobre l’entorn de desenvolupament descrit més avall. La construcció es gestiona amb GNU Make 4.4.1 mitjançant fitxers de construcció separats per component i per sistema operatiu, de manera que el motor, l’aplicació de proves i el projecte de proves unitàries es compilen de manera independent. El motor es genera com a biblioteca compartida i l’aplicació de proves s’hi enllaça.

3.4.2 API gràfica i eines associades

El renderitzador es construeix sobre Vulkan. Els *shaders* s’escriuen en GLSL i es compilen a SPIR-V amb `glslc` com a pas posterior a la construcció, atès que l’API només accepta aquest format intermedi [5]. La correcció de l’ús de l’API es verifica amb les capes de validació de Khronos, habilitades únicament a les compilacions de depuració.

Les eines de Vulkan difereixen entre els dos entorns: a l’equip A provenen dels paquets de la distribució i a l’equip B del SDK de LunarG. Aquesta diferència no afecta el codi generat, atès que en tots dos casos els *shaders* es compilen a SPIR-V abans de l’execució, però sí que explica

que el fitxer README del projecte indiqui la variable `VULKAN_SDK` com a requisit mentre que a l'entorn Linux no calgui definir-la.

Convé no barrejar tres versions que apareixen al llarg del document i que designen coses diferents: la de les eines de compilació de *shaders*, la de l'especificació citada com a font normativa —la 1.4.361— i la que cada dispositiu reporta en temps d'execució, recollida a la Taula 5. Aquesta darrera és una propietat del controlador i té conseqüències pràctiques que es discuteixen al capítol 6.

3.4.3 Control de versions i documentació

El control de versions s'ha fet amb Git 2.55.0, amb un historial de *commits* d'unitat funcional que és, ahora, el registre cronològic del desenvolupament. La documentació tècnica del projecte es manté en format Markdown dins del mateix repositori, i aquesta memòria s'ha redactat amb Typst 0.15.1. L'anàlisi del codi durant el desenvolupament s'ha recolzat en `clangd`.

3.4.4 Ús d'eines d'intel·ligència artificial

S'han emprat assistents basats en models de llenguatge durant el desenvolupament i la redacció, i convé declarar-ne l'ús amb precisió: no és el mateix delegar-hi una decisió que fer-los servir per a una tasca acotada. Els usos han estat els següents.

Diagnòstic de defectes En diversos punts del desenvolupament, el sistema de registre del motor no va arribar a capturar la causa d'un defecte, o la va descriure d'una manera que no permetia identificar-la amb facilitat. En aquests casos s'ha recorregut a aquestes eines per acotar-ne l'origen a partir del codi i del comportament observat. La correcció i la seva comprovació s'han fet sempre sobre el motor en execució.

Ordenació de les referències S'han fet servir per mantenir coherent l'aparell de citacions al llarg de la memòria, comprovar que cada entrada de la bibliografia es citava efectivament al text i verificar les dades de les entrades.

Revisió lingüística S'han emprat per revisar l'ortografia i la gramàtica del text i, sobretot, per assenyalar els punts on el registre o la terminologia variaven entre seccions, cosa difícil de detectar rellegant un document d'aquesta extensió.

Automatització de les mesures Els *scripts* que executen les repeticions de l'experiment del capítol 6 i en recullen la sortida s'han generat amb aquestes eines. No formen part del motor i no intervenen en el que es mesura: només llancen el binari amb cada configuració i desen els registres.

3.4.5 Maquinari de desenvolupament

L'entorn de desenvolupament i de mesura es detalla a la Taula 5. Les seves característiques no són un detall administratiu: el fet que la GPU sigui integrada i comparteixi memòria amb el sistema condiciona directament una part de les decisions del capítol 4 i és la premissa dels experiments del capítol 6.

Taula 5: Entorns de desenvolupament i de mesura.

	Equip A — GPU integrada	Equip B — GPU discreta
Processador	Intel Core i7-1255U (12a gen.), 10 nuclis / 12 fils	AMD Ryzen 7 4800HS, 8 nuclis / 16 fils

	Equip A — GPU integrada	Equip B — GPU discreta
GPU	Intel Iris Xe Graphics (ADL GT2), integrada	NVIDIA GeForce GTX 1660 Ti with Max-Q Design, dedicada
Arquitectura de memòria	Un únic munt; dos tipus de memòria alhora locals al dispositiu i visibles des de l'amfitrió	5,83 GiB de memòria dedicada, més una finestra de 214 MiB accessible des de l'amfitrió
Controlador	Mesa 26.2.1 (codi obert d'Intel)	NVIDIA 566.14
Versió de Vulkan del dispositiu	1.4.354	1.3.289
Famílies de cues	Una de sola per a gràfics i presentació	Diferenciades: gràfics 0, presentació 2
Sistema operatiu	Arch Linux, nucli 7.2.2	Windows 11 Pro 10.0.26200
Memòria del sistema	15,30 GiB	15,42 GiB
Compilador	Clang 22.1.8	Clang 12.0.0 (x86-64, MSVC)
Eines de Vulkan	Paquets de la distribució (shaderc 2026.3)	SDK de LunarG 1.4.313.1

La disponibilitat de dos equips amb arquitectures de memòria diferents no és una circumstància accessòria, sinó una condició que el treball aprofita de manera deliberada. L'equip A reporta el dispositiu com a `VK_PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU`: no disposa de memòria local dedicada, i els tipus de memòria visibles des del processador coincideixen en bona part amb els locals al dispositiu. L'equip B, amb GPU dedicada, presenta la separació clàssica entre les dues.

Aquesta diferència travessa tot el treball. És la que va fer aflorar el defecte de sincronització documentat a la secció d'aportacions pròpies, que només es manifesta sobre l'equip A; és la que dona sentit a l'advertència de la documentació de Mantle sobre no presuposar la visibilitat directa de la memòria [4]; i és la que permet que els experiments del capítol 6 siguin comparatius en lloc de descriptius, contrastant el comportament de les dues arquitectures en lloc de caracteritzar-ne una de sola.

4 Decisions de disseny

Les decisions que recull aquest capítol no es presenten com a solucions òptimes. En programari rarament n'hi ha cap que ho sigui en termes absoluts: hi ha opcions que encaixen millor amb uns objectius determinats i pitjor amb uns altres, i el que fa útil una decisió és haver-la presa sabent amb quins objectius havia d'encaixar. El criteri que ha guiat el projecte ha estat, doncs, escollir en funció dels objectius plantejats al capítol 1 —i, en algun cas, de l'interès per entendre un mecanisme concret— després d'haver considerat les alternatives disponibles.

Cada secció segueix la mateixa estructura: quin problema planteja la decisió, quines alternatives es van considerar, quin criteri es va aplicar i què s'hi va guanyar i què s'hi va perdre. L'últim punt hi consta sempre, perquè és el que permet jutjar si el criteri era encertat.

4.1 Vulkan com a API gràfica

4.1.1 Problema i alternatives

La primera decisió del projecte condiciona la resta: sobre quina API gràfica es construeix el renderitzador. El criteri previ que va delimitar el conjunt d'alternatives va ser que es tractés d'una API amb ús professional real a la indústria del videojoc, atès que un dels propòsits del treball és entendre les eines que s'hi fan servir. Aquest criteri va deixar fora opcions com WebGPU, que no es van arribar a considerar.

Van quedar, per tant, tres candidates: OpenGL, Vulkan i DirectX 12.

OpenGL era la més coneguda de partida, atès que s'ha tractat durant el grau. Ara bé, el seu ús en desenvolupament professional de videojocs ha anat disminuint, i —cosa més determinant per a aquest projecte— el seu model clàssic delega al controlador la gestió de riscos d'accés, la sincronització i l'assignació de memòria [2], que són precisament els mecanismes que el treball es proposa entendre. Una API que els resol de manera transparent en fa impossible l'estudi.

Descartada OpenGL, la tria quedava entre les dues API de control explícit d'ús generalitzat: Vulkan i DirectX 12.

4.1.2 Criteri i decisió

Convé ser explícit sobre un punt: qualsevol de les dues hauria servit. L'objectiu plantejat no és dominar una API concreta sinó entendre com funciona una API gràfica moderna de control explícit, i tant Vulkan com DirectX 12 responen a aquesta descripció i tenen ús professional consolidat. La decisió, doncs, no es va prendre sobre la superioritat tècnica d'una respecte de l'altra.

El factor que la va determinar va ser la flexibilitat quant al sistema operatiu. DirectX 12 està limitat a plataformes de Microsoft, mentre que Vulkan permet executar el mateix codi sobre Windows i Linux, cosa que encaixava amb l'objectiu de portabilitat plantejat al capítol 1.

Aquesta previsió es va acabar exercint de manera efectiva, i no només com a possibilitat teòrica. El projecte va començar sobre Windows —la capa de plataforma inicial és la d'aquest sistema— i posteriorment es va portar a Linux, que ha estat l'entorn de desenvolupament principal durant els darrers mesos. Aquest canvi d'entorn, que respon a circumstàncies alienes al disseny del motor, no hauria estat possible amb DirectX 12. És, per tant, un cas en què el criteri aplicat va resultar rellevant per raons que en el moment de decidir només es podien anticipar de manera genèrica.

La portabilitat que aquesta decisió proporciona no s'esgota, a més, en els dos sistemes operatius emprats. Vulkan és igualment accessible sobre les plataformes d'Apple, si bé cal precisar per quina via: macOS no dona suport natiu a Vulkan —el seu entorn gràfic propi és Metal—, sinó que aquest suport s'obté mitjançant MoltenVK, una implementació que desplega un subconjunt gairebé complet de Vulkan per sobre de Metal i que tradueix els *shaders* de SPIR-V al llenguatge d'ombreig de Metal [14]. Aquesta implementació dona suport a la versió 1.4 de l'API, que és la que el motor sol·licita. Incorporar macOS requeriria, per tant, escriure la capa de plataforma corresponent i la creació de superfície específica, però no afectaria la resta del renderitzador. Queda fora de l'abast del treball, tal com s'ha delimitat al capítol 1.

Aquesta manera d'entendre la decisió té reflex, també, en l'arquitectura del motor. El renderitzador se separa en una part independent de l'API i una implementació concreta, i el tipus enumerat que identifica la implementació activa preveu, a més de Vulkan, els casos d'OpenGL i DirectX. Incorporar una segona implementació requeriria escriure-la i afegir una branca a la funció que en fa la selecció, sense modificar ni la part independent de l'API ni el codi del joc. Aquesta separació es descriu en detall més endavant en aquest mateix capítol.

Convé no exagerar l'abast de cap d'aquestes dues afirmacions. Ni existeix una implementació per a una API alternativa ni una capa de plataforma per a macOS, i escriure qualsevol de les dues no seria un exercici menor; en el cas de macOS, caldria a més verificar quines restriccions imposa un subconjunt de portabilitat sobre les funcionalitats concretes que el motor utilitza. El que la decisió preserva no és la portabilitat immediata, sinó el fet que triar Vulkan no hagi tancat cap d'aquestes portes.

4.1.3 Cost de la decisió

El cost principal no prové de triar Vulkan en lloc de DirectX 12 —on hauria estat comparable— sinó d'haver optat per una API de control explícit en lloc d'una de clàssica, i convé enunciar-lo sense atenuants. El mateix consorci Khronos reconeix que l'API és verbosa i complexa, que exposa nombroses arestes i que hi ha molt per aprendre [3]. La conseqüència pràctica és que la quantitat de codi necessària per arribar a dibuixar el primer triangle és d'un ordre de magnitud superior a la que requeriria OpenGL, i que una part substancial del calendari descrit al capítol 3 es va destinar a construir infraestructura que no produeix cap resultat visible.

A canvi, l'API ofereix un sobrecost de controlador molt inferior, repartible entre múltiples fils d'execució, i un rendiment més previsible [3]. D'aquests avantatges, aquest treball només n'aprofita parcialment el primer i el tercer: el motor és d'un sol fil, de manera que la capacitat de repartir la feina entre nuclis —que és probablement el guany estructural més important de l'API— queda sense explotar. Aquesta limitació es discuteix al capítol 7.

4.2 C com a llenguatge

4.2.1 Problema i alternatives

L'elecció del llenguatge es va plantejar entre C i C++, que són les dues opcions realistes per a un motor que ha d'interoperar amb una API en C i mantenir control sobre la disposició de la memòria.

C++ ofereix genèrics, gestió automàtica de recursos mitjançant el patró de concessió de recursos en la inicialització, sobrecàrrega d'operadors i una biblioteca estàndard extensa [15]. C, per la seva banda, és un llenguatge de superfície considerablement més reduïda [16]. Aquest patró, en particular, permet adquirir recursos en el constructor i alliberar-los en el destructor, cosa que elimina les operacions d'assignació explícites del codi general [15].

4.2.2 Una precisió sobre el disseny orientat a dades

L'interès per les tècniques de disseny orientat a dades va motivar inicialment la inclinació cap a C. Convé, però, precisar l'abast d'aquest argument, perquè formulat de manera descuidada no se sosté.

El disseny orientat a dades és la pràctica de dissenyar programari desenvolupant transformacions sobre dades ben formades, on el criteri del que és ben format ve determinat pel maquinari objectiu i pels patrons de transformació que s'hi han d'aplicar [17]. És, per tant, una qüestió de disposició de les dades i de coneixement del maquinari, i no d'elecció de llenguatge: la font de referència afirma explícitament que no està arrelat en un sol llenguatge [17].

Aquesta mateixa font assenyala que el llenguatge que més se'n beneficia és C++. La construcció de la frase és, però, paral·lela a la que fa servir per al maquinari, on sosté que el que més se'n beneficia és aquell que presenta colls d'ampolla desequilibrats [17]; és a dir, aquell on el problema és més acusat i, en conseqüència, el marge de millora més gran. Llegida en aquest sentit, l'afirmació no diu que C++ sigui el vehicle més adequat per al disseny orientat a dades, sinó que és on hi ha més a guanyar-hi.

Aquesta lectura és coherent amb l'objecte real de la crítica que l'obra desenvolupa, que no és cap llenguatge concret sinó el disseny orientat a objectes com a metodologia: el retret central és que acobla el domini del problema amb la implementació, i que aquest acoblament genera una inèrcia que dificulta l'adaptació quan les dades canvien [17]. C++ no obliga a programar així, i el disseny orientat a dades hi és plenament practicable.

Cal admetre, tanmateix, un matis que la distinció anterior no cobreix del tot: si bé el llenguatge no imposa aquell estil, sí que en facilita l'adopció. Els constructors i destructors que s'executen implícitament en entrar i sortir d'àmbit, el despatx virtual i els contenidors genèrics d'objectes fan que l'organització que la crítica desaconsella sigui també el camí de mínima resistència. La distinció útil, doncs, no és entre llenguatges que permeten el disseny orientat a dades i llenguatges que no, sinó entre llenguatges que hi empenyen i llenguatges que empenyen cap a una altra banda.

En qualsevol cas, aquest treball no fa servir el disseny orientat a dades com a argument per triar C. El que sí que va aportar l'interès per aquestes tècniques va ser orientar l'atenció cap a la disposició explícita de les dades en memòria, i és aquesta preocupació —no la metodologia en si— la que té conseqüències sobre l'elecció del llenguatge.

4.2.3 Criteri i decisió

El criteri efectiu és l'explicitud. C no disposa de constructors ni destructors implícits, ni de gestió automàtica de recursos, ni d'assignacions ocultes rere abstraccions. Tota reserva de memòria, tot alliberament i tota còpia són visibles al lloc on succeeixen, i el llenguatge no ofereix cap mecanisme per delegar-ne la responsabilitat. Programar-hi obliga, per tant, a un tracte deliberat amb la memòria en cada punt on s'hi accedeix.

En un motor construït sobre una API que trasllada deliberadament a l'aplicació la gestió de memòria, la sincronització i el cicle de vida dels recursos —tal com s'ha exposat al capítol 2—, aquesta correspondència entre el model del llenguatge i el model de l'API té valor pràctic: allò que cal fer explícit a la GPU també ho és al codi que la governa. En un treball on la disposició i el cicle de vida de les dades constitueixen l'objecte d'estudi, un llenguatge que els gestioni en nom del programador n'obstaculitza l'anàlisi, encara que permetés obtenir el mateix resultat amb menys codi.

Convé precisar l'ordre en què es van prendre aquestes decisions, atès que el capítol 3 documenta l'ús d'una implementació de referència també escrita en C. L'elecció del llenguatge va ser prèvia: la cerca de materials de referència es va fer un cop presa, i el fet que estiguessin escrits en C va ser un criteri de selecció d'aquests materials i no un factor que intervingués en la decisió.

4.2.4 Cost de la decisió

L'absència de genèrics obliga a implementar els contenidors mitjançant macros i aritmètica de mides en temps d'execució, amb la pèrdua de comprovació de tipus que això comporta. La gestió manual de recursos trasllada al programador la responsabilitat que el patró de concessió de recursos en la inicialització automatitza, amb el risc de fuites i alliberaments duplicats que se'n deriva. I la biblioteca estàndard, molt més reduïda, obliga a implementar funcionalitat que en C++ seria immediata, com ara la manipulació de cadenes o les estructures de dades associatives.

Aquests costos són reals i el motor els paga. La decisió es defensa perquè l'objectiu del treball no és minimitzar l'esforç d'implementació, sinó fer visible el comportament que un motor de jocs ha de controlar.

4.3 El motor com a biblioteca compartida i *framework*

4.3.1 Problema i alternatives

Un motor i el joc que s'hi construeix han de repartir-se el control del flux d'execució, i cal decidir quina de les dues parts el posseeix. Hi ha dues maneres habituals de resoldre-ho [1]. Una biblioteca és un conjunt de funcions i classes que l'aplicació invoca com li convé: ofereix màxima flexibilitat al programador, a canvi que aquest hagi d'entendre com utilitzar-la correctament. Un *framework*, en canvi, és una aplicació parcialment construïda en què el bucle principal ja està escrit però buit, i que el programador completa proporcionant les implementacions que hi manquen; en aquest cas té poc o cap control sobre el flux general.

4.3.2 Criteri i decisió

El motor s'ha construït com a *framework*. La funció `main` resideix dins de la biblioteca del motor i no al costat del joc, i és el motor qui condueix la seqüència d'arrencada, el bucle principal i el tancament.

El joc, per la seva banda, ha de proporcionar una funció que construeixi la seva instància i n'ompli una estructura amb quatre punters a funció —inicialització, actualització, renderitzat i resposta a canvis de mida— juntament amb la configuració de l'aplicació i un punter a l'estat propi del joc, del qual el motor no coneix l'estructura interna. Abans de continuar, el punt d'entrada verifica que els quatre punters existeixin i avorta amb un diagnòstic si no és així.

El criteri que va portar a aquesta decisió és que el bucle principal conté ordenacions que no són negociables i que no convé que cada joc hagi de tornar a resoldre. El capítol 5 en detalla un cas concret: l'actualització de l'estat d'entrada ha de ser l'última operació de cada iteració, perquè és la que converteix l'estat actual en estat previ i permet consultar transicions de tecla a la iteració següent. Fer-ho en un altre punt no produeix cap error visible, però trenca silenciosament la detecció de polsacions. Ordenacions d'aquesta mena justifiquen que el bucle sigui responsabilitat del motor.

4.3.3 Cost de la decisió

El cost és el que s'atribueix habitualment a aquest enfocament: el joc perd el control del flux [1]. Un joc que necessités una estructura de bucle diferent —per exemple, per integrar-se dins d'una

aplicació amfitriona amb el seu propi bucle d'esdeveniments— no podria fer-ho sense modificar el motor.

Hi ha, a més, una conseqüència tècnica de la implementació escollida que convé fer explícita: la funció `main` es proporciona dins d'una capçalera que el codi del joc inclou, de manera que és la unitat de compilació del joc la que efectivament la compila. És una solució que funciona i que manté el punt d'entrada sota control del motor, però que implica que la capçalera només es pot incloure des d'un únic fitxer del projecte del joc.

4.4 Gestió explícita de memòria

Aquesta és la decisió més característica del motor i la que millor il·lustra la tesi que travessa el treball: que les fonts consultades, tot i respondre a preocupacions diferents, convergeixen en la mateixa solució.

4.4.1 Problema i alternatives

Cada subsistema del motor manté un estat intern que ha de viure mentre duri l'execució. Cal decidir qui el reserva, on resideix i quan es allibera.

L'opció immediata és que cada subsistema reservi el seu estat pel seu compte mitjançant l'assignador general, i que l'alliberi en tancar-se. És senzilla i no requereix cap coordinació, però reparteix l'estat del motor per tot el munt de memòria, obliga cada subsistema a dependre d'un assignador concret i no dona al motor cap control sobre la ubicació ni sobre el moment de l'alliberament.

4.4.2 La convergència de dues exigències

La solució adoptada respon a dues exigències de procedència independent.

La primera prové de l'arquitectura de motors. Com s'ha exposat al capítol 2, les dependències entre subsistemes imposen un ordre d'arrencada, i la recomanació és fer aquest ordre explícit mitjançant funcions d'arrencada i tancament que s'invoquen des d'un únic lloc, en lloc de confiar-lo als mecanismes implícits del llenguatge [1].

La segona prové de l'API gràfica. Vulkan no reserva memòria en nom de l'aplicació, i això li imposa una convenció de crida característica: per obtenir una llista d'elements, la primera crida es fa amb el punter de destinació nul i retorna el nombre d'elements disponibles; la segona, amb l'espai ja reservat per l'aplicació, l'omple [5]. Qui reserva l'emmagatzematge ha de saber-ne primer la mida.

El patró d'inicialització del motor és la composició d'ambdues. Cada funció d'inicialització s'invoca dues vegades: la primera amb el punter d'estat nul, cas en què es limita a informar de la mida que necessita; entre les dues crides, l'aplicació reserva aquesta quantitat de memòria; i la segona crida ja rep el bloc i hi construeix l'estat. Aquesta seqüència es repeteix per a cada subsistema, en l'ordre que les dependències imposen, dins d'una única funció. El Codi 1 en mostra les dues bandes.

```

// Costat del subsistema: la mateixa funció respon les dues preguntes.
b8 subsistema_initialize(u64* memory_requirement, void* state) {
    *memory_requirement = sizeof(subsistema_state);
    if (state == 0) {
        return true;          // primera crida: només informa de la mida
    }
    state_ptr = state;        // segona crida: rep el bloc i s'hi instal·la
    /* ... inicialització efectiva ... */
    return true;
}

// Costat de l'aplicació: preguntar, reservar, inicialitzar.
subsistema_initialize(&mida, 0);
estat = linear_allocator_allocate(&app_state->systems_allocator, mida);
subsistema_initialize(&mida, estat);

```

Codi 1: Patró d'inicialització en dues crides. La primera invocació informa de la mida de l'estat; l'aplicació reserva aquesta quantitat de l'assignador lineal i la lliura a la segona. El subsistema no decideix, doncs, ni on resideix el seu estat ni quan s'allibera.

La memòria surt d'un assignador lineal, que reserva un bloc de 64 MiB en iniciar-se i el reparteix de manera incremental sense alliberar-ne mai peces individuals. L'estat de tots els subsistemes acaba, doncs, en una única regió contiguous amb un cicle de vida igual al del procés.

4.4.3 Criteri i decisió

El resultat és que ni la ubicació de l'estat ni el seu moment d'alliberament són decisions del subsistema, sinó de qui el condueix. Un subsistema que no reserva la seva pròpia memòria no pot imposar cap assignador a la resta del motor, no pot fragmentar el munt i no pot sobreviure al bloc que se li ha lliurat. S'obté, a més, una propietat associada a l'ús d'assignadors d'aquesta mena: la proximitat en memòria de l'estat que s'utilitza conjuntament [1].

4.4.4 Cost de la decisió

Aquest patró té tres costos que convé enunciar.

El primer és que cada subsistema desa el bloc rebut en un punter estàtic de la seva unitat de compilació, cosa que fa ergonòmic el patró —cap funció no ha de rebre un paràmetre de context— però limita el motor a una única instància de cada subsistema per procés. No hi pot haver dos renderitzadors ni una cua d'esdeveniments per fil d'execució.

El segon és que la reserva no imposa cap alineació: retorna el següent byte lliure. Els estats dels subsistemes queden correctament alineats per la mida de les estructures que els precedeixen, no per garantia de l'assignador. És una limitació coneguda i es discuteix al capítol 7.

El tercer és que la xifra de 64 MiB descriu l'estat dels subsistemes, però no la petjada real del motor: els subsistemes que reserven memòria durant la seva inicialització —el registre obre un fitxer, el sistema d'esdeveniments crea vectors dinàmics— ho fan fora d'aquest bloc.

4.5 Separació entre la part independent de l'API i la implementació

4.5.1 Problema i alternatives

Perquè la decisió documentada a la secció sobre l'API gràfica no quedés en una declaració d'intencions, calia que el motor no assumís Vulkan arreu. Es tracta de decidir on se situa la frontera entre allò que qualsevol renderitzador ha de fer i allò que només té sentit sobre una API concreta.

4.5.2 Criteri i decisió

La frontera es materialitza en una estructura de punters a funció que la part independent de l'API invoca i que una funció de selecció omple segons la implementació activa, tal com recull el Codi 2.

```
typedef struct renderer_backend {
    u64 frame_number;

    b8 (*initialize)(struct renderer_backend*, const char* application_name);
    void (*shutdown)(struct renderer_backend* backend);
    void (*resized)(struct renderer_backend* backend, u16 width, u16 height);

    b8 (*begin_frame)(struct renderer_backend* backend, f32 delta_time);
    void (*update_global_world_state)(mat4 projection, mat4 view, vec3
view_position,
                                vec4 ambient_color, vec4 light_direction,
                                vec4 light_color, i32 mode);
    void (*update_global_ui_state)(mat4 projection, mat4 view, i32 mode);
    b8 (*end_frame)(struct renderer_backend* backend, f32 delta_time);

    b8 (*begin_renderpass)(struct renderer_backend* backend, u8
renderpass_id);
    b8 (*end_renderpass)(struct renderer_backend* backend, u8 renderpass_id);

    void (*draw_geometry)(geometry_render_data data);

    void (*create_texture)(const u8* pixels, struct texture* texture);
    void (*destroy_texture)(struct texture* texture);
    b8 (*create_material)(struct material* material);
    void (*destroy_material)(struct material* material);
    b8 (*create_geometry)(geometry* geometry, /* ... */);
    void (*destroy_geometry)(geometry* geometry);
} renderer_backend;
```

Codi 2: La frontera entre la part independent de l'API i la seva implementació. Cap paràmetre és un objecte de Vulkan: les operacions parlen de *frames*, de geometries i de materials.

El criteri que governa aquesta llista és el nivell d'abstracció. Cap dels paràmetres és un objecte de l'API gràfica: no hi ha cap entrada per crear una canonada, ni per vincular un conjunt de descriptors, ni per reservar memòria de dispositiu. Aquesta elecció és deliberada. Una interfície de gra fi que exposés els conceptes de Vulkan hauria estat més flexible, però hauria traslladat el model d'aquesta API a la part que se suposava independent, i qualsevol implementació alternativa hauria hagut d'emular-lo.

La conseqüència és que la part independent pot mantenir les matrius de projecció i de vista i la informació d'il·luminació sense saber què és un conjunt de descriptors, i que tot allò específic de l'API —memòria de dispositiu, *command buffers*, sincronització, disposicions d'imatge— queda confinat rere aquestes funcions.

Convé assenyalar que aquesta frontera s'ha desplaçat durant el desenvolupament. Les dues operacions que delimiten una passada de renderitzat es van incorporar en generalitzar el motor per admetre'n més d'una, i introdueixen a la part independent un concepte manllevat de l'API gràfica. La concessió està parcialment continguda —la passada s'identifica amb un enter opac i no amb cap objecte de Vulkan, de manera que una implementació alternativa podria interpretar-lo com li convingués—, però és una concessió. Il·lustra que una frontera d'aquesta mena no es

fixa d'una vegada sinó que cedeix a mesura que el sistema creix, i que mantenir-la exigeix decidir cada cop si la pressió es resisteix o s'accepta.

4.5.3 Cost de la decisió

Els punters a funció no reben cap paràmetre de context. La implementació manté, per tant, el seu propi estat de manera estàtica, cosa que simplifica les signatures a canvi d'impedir que hi hagi dues implementacions actives simultàniament. Per a un motor que selecciona la implementació en temps de construcció, la limitació no té conseqüències pràctiques.

Hi ha, a més, una fuita coneguda i documentada al codi. La funció que estableix la matriu de vista s'exporta fora del motor perquè l'aplicació de proves hi pugui empènyer la matriu de la càmera, i la capçalera la marca explícitament com una solució provisional. La via correcta seria que el joc escrivís un estat de més alt nivell i que el renderitzador el llegís del paquet de renderitzat que ja rep a cada *frame*, però aquest paquet encara no transporta informació d'escena. És una excepció reconeguda al criteri que la resta de la secció defensa.

4.6 Model de recursos: referències, identificadors i generacions

4.6.1 Problema i alternatives

Els recursos que el motor carrega des de disc —textures i, posteriorment, materials— es comparteixen entre diverses parts de l'escena i tenen un cost d'ocupació a la GPU. Cal decidir quan se'n reclama la memòria.

Aquesta qüestió es planteja habitualment com la gestió del cicle de vida dels recursos, i cada recurs hi té requisits propis: alguns han de romandre en memòria durant tota la partida, altres tenen un cicle de vida lligat al d'un nivell concret, i altres encara més breu [1]. La solució establerta és el recompte de referències, amb la descàrrega del recurs quan el recompte arriba a zero [1].

4.6.2 Criteri i decisió

El motor implementa aquest esquema mitjançant una taula de dispersió que associa el nom del recurs amb una estructura que en registra el recompte de referències, un identificador i un indicador d'alliberament automàtic. Adquirir un recurs n'incrementa el recompte i, si encara no existeix, en desencadena la càrrega; alliberar-lo el decrementa.

L'indicador d'alliberament automàtic és el que aporta la política de cicle de vida. Un recurs adquirit amb aquest indicador desactivat es comporta com un actiu global [1]: roman carregat encara que el recompte arribi a zero. Amb l'indicador actiu, es descarrega en deixar de tenir referències. És una versió d'un sol bit d'una taxonomia de cicles de vida més detallada [1], però n'expressa la distinció essencial.

A aquest esquema s'hi afegeix un segon mecanisme. Cada recurs manté, a més de l'identificador, un camp de generació que s'incrementa cada vegada que el contingut es torna a carregar. Un punter per si sol no permet saber que allò que apunta ha estat substituït; la parella formada per l'identificador i la generació sí que ho permet [18].

Aquest mecanisme té una utilitat concreta al renderitzador. Els conjunts de descriptors que vinculen una textura a un *shader* han de reescriure's quan la textura canvia, però reescriure'ls a cada *frame* seria innecessari. La implementació desa, per a cada descriptor, l'identificador i la generació de la textura que hi té vinculada, i només emet l'operació d'actualització quan algun dels dos difereix del valor actual del recurs.

4.6.3 Cost de la decisió

El recompte de referències és manual: si una part del motor adquireix un recurs i no l'allibera, aquest no es descarregarà mai. El llenguatge no ofereix cap mecanisme que ho previngui, cosa que és conseqüència directa de la decisió documentada a la secció sobre el llenguatge.

Hi ha, a més, una limitació estructural. Aquest esquema s'implementa de manera independent per a cada tipus de recurs, de manera que el sistema de textures i el de materials repeteixen la mateixa estructura —taula de dispersió, vector de registres, recompte i indicador d'alliberament— sense compartir-ne cap. Falta la capa genèrica de gestió de recursos que unificaria l'accés a qualsevol tipus d'actiu [1]. Es tracta d'una de les línies de continuació que es recullen al capítol 7.

El mateix esquema s'aplica avui a tres tipus de recurs —textures, materials i geometries—, cosa que fa que la mancança descrita al paràgraf anterior sigui més visible: la repetició no afecta dos sistemes sinó tres.

5 Disseny i implementació

5.1 Arquitectura general i cicle de vida

5.1.1 Estructura

El motor es distribueix com a biblioteca compartida i l'aplicació que l'exercita s'hi enllaça. La correspondència entre els seus mòduls i les capes del model de referència ja s'ha establert a la Taula 1; aquesta secció en descriu el funcionament dinàmic, és a dir, què succeeix en arrencar i què succeeix a cada *frame*.

5.1.2 Seqüència d'arrencada

La funció que construeix l'aplicació reserva un bloc de 64 MiB i inicialitza onze subsistemes en un ordre fix, segons el patró descrit al capítol 4. L'ordre no és arbitrari: cada posició respon a una dependència concreta, i la Taula 6 les recull.

Taula 6: Ordre d'inicialització dels subsistemes i dependència que el motiva.

	Subsistema	Motiu de la posició
1	Esdeveniments	L'entrada hi publica; ha d'existir abans que ningú s'hi subscrigui.
2	Memòria	Només instal·la el comptador: les reserves funcionen abans i es declaren en aquest punt.
3	Registre	Depèn del sistema de fitxers, que no requereix inicialització, i de la sortida per consola de la plataforma, que funciona sense finestra.
4	Instrumentació	Ha de precedir tot allò que pugui mesurar. Només depèn del rellotge absolut.
5	Entrada	Requereix el sistema d'esdeveniments.
6	Plataforma	Crea la finestra.
7	Recursos	Estableix l'arrel dels actius i registra els carregadors.
8	Renderitzador	Necessita la finestra per crear la superfície de presentació.
9	Textures	Necessita el renderitzador per crear recursos al dispositiu.
10	Materials	Adquireix textures.
11	Geometria	Adquireix materials.

Les quatre darreres posicions formen una cadena de dependències estricta que il·lustra el principi enunciat al capítol 2: la geometria depèn dels materials, que depenen de les textures, que depenen del renderitzador. Invertir qualsevol d'aquests parells produiria una dependència circular.

Val la pena assenyalar dues posicions que semblen arbitràries i no ho són. El sistema de memòria ocupa la segona posició tot i que abans d'arribar-hi ja s'han reservat l'estat de l'aplicació i el bloc de l'assignador; això funciona perquè la inicialització d'aquest subsistema només instal·la el comptador d'ocupació i la reserva de memòria no en depèn. Que les reserves anteriors quedin per sota del comptador és una conseqüència que §5.2 documenta i resol. El registre ocupa la tercera tot i dependre de la plataforma, perquè la funció d'escriptura per consola que utilitza no requereix que la finestra existeixi.

5.1.3 Cicle d'un *frame*

El bucle principal executa, a cada iteració, la seqüència que recull la Figura 3.

1	Plataforma	Drena els esdeveniments del sistema, que alimenten l'entrada, que publica esdeveniments del motor.
2	Rellotge	Calcula el pas de temps de la iteració.
3	Actualització	Crida la funció del joc; el codi de prova hi mou la càmera.
4	Renderitzat	Crida la funció de renderitzat del joc, avui buida.
5	Paquet	Construeix el paquet amb les geometries de món i d'interfície.
6	Dibuix	Enregistra i envia la feina de renderitzat al dispositiu.
7	Entrada	Copia l'estat actual d'entrada a l'estat previ.

Figura 3: Seqüència d'una iteració del bucle principal.

L'últim pas mereix atenció perquè és l'exemple concret que justifica la decisió documentada al capítol 4 sobre qui posseeix el bucle. La còpia de l'estat d'entrada ha de ser l'última operació de la iteració: és la que converteix l'estat actual en previ i, per tant, la que permet distingir una tecla que s'acaba de prémer d'una que ja ho estava. Situar-la en qualsevol altre punt no produeix cap error visible ni cap advertència, però fa que la detecció de transicions deixi de funcionar de manera silenciosa. És una ordenació que el motor ha de garantir i que no convé confiar a cada joc.

La construcció del paquet de renderitzat és, avui, codi provisional marcat com a tal: recull una geometria de món i una d'interfície fixes. És el punt on hauria d'encaixar una representació d'escena, i la seva absència es discuteix al capítol 7.

5.2 Gestió de memòria

La gestió de memòria del motor es reparteix en tres nivells amb propòsits diferents: una capa de reserva etiquetada que fa visible on van els bytes, un assignador lineal que decideix on resideix l'estat dels subsistemes, i la memòria del dispositiu, que l'API no gestiona en nom de l'aplicació. A aquests tres s'hi afegeix la capa d'instrumentació que els mesura.

5.2.1 Reserva etiquetada

Totes les reserves del motor passen per una funció pròpia que embolcalla la de la plataforma i hi afegeix una etiqueta de categoria: vectors dinàmics, cadenes, textures, renderitzador, escena i altres, fins a divuit. La capa manté el total acumulat per etiqueta i el nombre total de reserves efectuades.

L'objectiu no és el rendiment —la reserva subjacent és la del sistema— sinó la visibilitat. Permet respondre en qualsevol moment a on són els bytes, i el comptador de reserves permet detectar reserves accidentals dins del bucle principal, que és el diagnòstic que la bibliografia recomana per a aquest cas [1].

La capa no manté cap metadada per reserva individual, cosa que té una conseqüència visible a la interfície: qui allibera ha de tornar a indicar la mida. És el preu de no afegir una capçalera a cada bloc.

5.2.2 L'assignador lineal

El segon nivell és un assignador lineal sobre un bloc de mida fixa: reparteix adreces de manera incremental i no allibera mai peces individuals, només el bloc sencer. Admet dos modes de propietat, segons si reserva el bloc ell mateix o opera sobre un que se li lliura.

L'aplicació en crea un de 64 MiB en arrencar i hi col·loca l'estat dels onze subsistemes mitjançant el patró descrit al capítol 4. El resultat és que tot l'estat de llarga durada del motor resideix en una regió contigua amb un cicle de vida igual al del procés.

Aquesta estructura correspon a l'assignador de pila que descriu la bibliografia [1], [19], amb una funcionalitat menys: la possibilitat d'obtenir un marcador de la posició actual i, més enda-

vant, retornar-hi alliberant de cop tot el que s'hagi reservat des d'aleshores. Aquesta absència no afecta l'ús actual —estat que viu tota l'execució— però impedeix l'ús que la mateixa font descriu a continuació: un assignador de *frame*, que es reinicia a cada iteració i permet reserves temporals a cost nul [1]. Es recull com a línia de continuació.

5.2.3 Memòria de dispositiu

El tercer nivell és el que l'API deixa explícitament a l'aplicació. Com s'ha exposat al capítol 2, el dispositiu exposa diversos *heaps* amb propietats diferents i l'aplicació ha de consultar-les en lloc de presuposar-les [5].

El motor obté la memòria del dispositiu mitjançant el patró de búfer intermedi: reserva un búfer visible des del processador, hi copia les dades, ordena una còpia cap a un búfer local al dispositiu i destrueix l'intermedi. El mateix procediment, amb una transició addicional de disposició d'imatge, porta els píxels d'una textura fins a la seva imatge.

Sobre el maquinari descrit al capítol 3 aquest procediment mereix una consideració que el capítol 6 examina amb mesures: en una arquitectura de memòria unificada els tipus de memòria local al dispositiu i visible des del processador coincideixen en bona part, de manera que la còpia intermèdia pot resultar innecessària.

La reserva de memòria de dispositiu es fa avui en blocs grans i fixos per als búfers de vèrtexs i d'índexs, sense subassignador. La documentació de referència adverteix que les reserves individuals estan limitades en nombre i recomana precisament reservar blocs grans i subassignar-hi a dins [4], [20]; el motor en fa la primera meitat i no la segona.

5.2.4 Instrumentació

Sobre aquests tres nivells hi ha una capa que mesura el comportament del motor sense recórrer a eines externes, seguint la descripció de perfilatge integrat i d'estadístiques de memòria de la bibliografia [1].

El seu funcionament és delimitar amb un nom les fases del *frame* i acumular el temps de cadascuna. Una secció visitada diverses vegades dins d'un mateix *frame* hi reporta el total i no l'última visita, cosa que importa per a operacions que poden repetir-se.

Les mesures no es reporten per *frame* sinó sobre una finestra mòbil de cent vint mostres, de la qual s'obtenen mitjana, mínim i màxim. La raó és que el temps d'un sol *frame* és majoritàriament soroll; una finestra mòbil dona xifres estables i, a diferència de reiniciar l'acumulador periòdicament, no introdueix cap discontinuïtat.

El disseny va requerir una segona estructura que no estava prevista inicialment. Les càrregues de recursos cap al dispositiu succeeixen gairebé totes durant l'arrencada, abans que existeixi cap *frame*, de manera que la finestra les reportava com a zero. La capa manté, per tant, acumuladors de vida sencera al costat de la finestra: temps total i nombre d'execucions per secció, independents del bucle. La distinció resulta ser la que separa les mesures de cadència de les mesures d'operacions puntuals, i el capítol 6 fa servir totes dues.

L'informe s'emet un cop per segon al registre, de manera que redirigir la sortida a un fitxer n'hi ha prou per capturar una sessió de mesura.

5.2.5 Mesura de temps de dispositiu

El rellotge de l'amfitrió no serveix per saber quant triga la GPU. Com que l'enviament de feina retorna immediatament, cronometrar-lo mesura el que el processador dedica a enregistrar i enviar, no el que el dispositiu dedica a executar. Obtenir la segona magnitud requereix un

mecanisme de l'API: escriure marques de temps dins del flux d'ordres i llegir-les quan s'han completat [5].

El motor reserva dues consultes per cada *frame* en vol i n'escriu una en obrir el *command buffer* i una altra just abans de tancar-lo. La diferència entre ambdues, multiplicada pel període de tic que el dispositiu declara, dona el temps d'execució.

Dos aspectes d'aquesta implementació mereixen comentari.

El primer és la comprovació de suport, que segueix el criteri establert a la secció sobre supòsits del dispositiu. El nombre de bits vàlids de marca de temps es reporta per família de cues i pot ser zero, cas en què les marques escrites en aquella cua no tenen cap valor. El motor el consulta abans de crear la reserva de consultes i, si no hi ha suport, ho registra i continua sense mesura de dispositiu: es tracta d'un diagnòstic, no d'una funcionalitat de la qual depengui el renderitzat. Sobre el maquinari de desenvolupament, un tic equival a 52,08 nanosegons.

El segon és quan es poden llegir els resultats. Una consulta només es pot llegir quan la feina que la va escriure ha acabat, i comprovar-ho exigiria una sincronització que alteraria precisament allò que es vol mesurar. La solució aprofita una espera que el motor ja fa: com s'explica a la secció sobre sincronització, cada iteració comença esperant la tancada del seu *frame* lògic, i aquesta espera garanteix que la feina enviada anteriorment en aquella mateixa ranura ha finalitzat. Els resultats es llegeixen just després, sense afegir cap sincronització nova.

La conseqüència és que la xifra obtinguda correspon al *frame* enviat tantes iteracions enrere com *frames* en vol permeti el motor, i no al que s'inicia. Sobre una finestra de cent vint mostres aquest desfasament és irrellevant, però convé tenir-lo present si mai s'interpreten mesures individuals.

5.2.6 Limitacions i defectes detectats

El model té dues limitacions que persisteixen i dos defectes que es van detectar en documentar-lo i que s'han corregit. Les primeres afecten el disseny dels assignadors; els segons, la fiabilitat de l'instrument de mesura i la correcció d'una inicialització.

Cap dels dos assignadors alinea les reserves: retornen l'adreça següent. Els estats dels subsistemes queden alineats per la mida de les estructures que els precedeixen i no per garantia de l'assignador. La bibliografia dedica un apartat a per què això és un problema de correcció en algunes arquitectures i de rendiment en la resta [1], [21], i l'API gràfica és estricta en aquest punt del costat del dispositiu, on exigeix que els desplaçaments siguin múltiples d'una alineació que el controlador reporta [5]. El costat de l'amfitrió del motor és, doncs, el laxe. El codi ho té marcat en dos punts.

L'assignador retorna un punter nul en exhaurir-se i ho registra, però la funció que en reparteix l'estat no en comprova el valor abans de lliurar-lo al subsistema. Exhaurir els 64 MiB no produiria un diagnòstic sinó un accés a memòria nul·la.

El primer dels dos defectes afectava el comptador d'ocupació, i es va detectar llegint el seu propi informe. Tant l'estat de l'aplicació com el bloc de 64 MiB es reserven mitjançant la funció de reserva del motor abans que el subsistema de memòria s'inicialitzi —n'és l'ordre descrit a la Taula 6—, i aquesta funció només anota l'estadística si el comptador ja existeix. L'informe emès just abans d'entrar al bucle principal declarava, per tant, poc més de quatre quilobytes, quan el procés en tenia reservats seixanta-quatre megabytes.

L'ordre no és arbitrari ni es pot invertir sense més: l'estat del comptador resideix dins del bloc que hauria de comptabilitzar, de manera que el bloc ha d'existir abans que el comptador. És una dependència circular inherent al model d'inicialització en dues crides descrit a §5.1, i qualsevol correcció ha de triar quin dels dos extrems trenca. Situar l'estat del comptador fora de l'assignador —en memòria estàtica o en una reserva pròpia— l'excloria del model que la resta del motor segueix, i per una sola excepció.

La correcció adoptada manté l'ordre i declara les dues reserves un cop el comptador és viu, mitjançant una funció que registra una reserva ja efectuada sense fer-ne cap. El Codi 3 en mostra les crides, situades immediatament després de la segona inicialització del subsistema de memòria.

```
memory_system_initialize(&app_state->memory_system_memory_requirement, 0);
app_state->memory_system_state = linear_allocator_allocate(
    &app_state->systems_allocator,
    app_state->memory_system_memory_requirement);
memory_system_initialize(&app_state->memory_system_memory_requirement,
    app_state->memory_system_state);

// The two reservations above had to happen before the counter existed, so
// declare them now. Without this the report would show a few kilobytes
// while the process holds the whole systems block.
hmemory_account_untracked(sizeof(application_state), MEMORY_TAG_APPLICATION);
hmemory_account_untracked(systems_allocator_total_size,
    MEMORY_TAG_LINEAR_ALLOCATOR);
```

Codi 3: Declaració de les dues reserves anteriors al comptador. La funció no reserva memòria: només n'anota una que ja s'ha produït, de manera que l'informe cobreix tot el que el motor ha demanat al sistema operatiu.

Amb la correcció aplicada, l'informe previ al bucle principal atribueix 64 MiB a l'etiqueta de l'assignador lineal, 264 B a l'estat de l'aplicació i poc més de quatre quilobytes a les reserves que els subsistemes efectuen durant la seva pròpia inicialització. La xifra passa a ser interpretable: recull tot el que el motor ha demanat al sistema operatiu.

Continua sense ser la petjada del procés, i convé no confondre-les. Per una banda, els 64 MiB són la capacitat reservada per a l'estat dels subsistemes i no la part que se n'ocupa efectivament, que l'assignador coneix però no publica. Per l'altra, el controlador de l'API gràfica reserva memòria pel seu compte, fora de qualsevol camí que el motor instrumenti.

El segon defecte és de naturalesa diferent i no té res a veure amb el comptador. La inicialització del sistema d'esdeveniments posava a zero el seu estat passant la mida del punter en lloc de la mida de l'estructura, de manera que netejava vuit bytes d'una taula de cent vint-i-vuit quilobytes. El defecte no s'havia manifestat mai, i la raó per la qual no ho feia és instructiva: la funció de reserva del motor lliura la memòria ja neta, i l'assignador lineal no reutilitza mai cap regió, de manera que l'estat era zero la primera vegada que s'utilitzava. El codi era incorrecte i funcionava perquè una propietat d'un altre component el cobria. S'ha corregit passant la mida de l'estructura.

5.3 Sincronització CPU/GPU i el bucle de *frame*

5.3.1 El problema

Com s'ha exposat al capítol 2, l'enviament de feina a una cua retorna el control a l'aplicació abans que la feina s'executi, i no hi ha cap restricció d'ordenació implícita entre operacions de cues diferents [5]. El renderitzador, per tant, no dibuixa: enregistra ordres i les envia. Tot allò que hagi de succeir en un ordre determinat ha de ser ordenat explícitament per l'aplicació.

L'API distingeix dues primitives per fer-ho [22], i la distinció és precisament la que determina quina s'utilitza a cada punt. Una tanca introdueix una dependència d'una cua cap a l'amfitrió: el processador hi espera per saber que la feina del dispositiu ha acabat [5]. Un semàfor introdueix una dependència entre operacions de cua: el dispositiu hi espera per saber que una altra feina seva ha acabat [5].

5.3.2 Les tres ordenacions

A cada *frame* cal garantir tres coses diferents, i el motor manté quatre vectors d'objectes de sincronització per fer-ho. La Taula 7 els recull.

Taula 7: Objectes de sincronització i criteri de dimensionament.

Vector	Mida	Funció
Tanques de <i>frame</i> en vol	<i>frames</i> en vol	El processador hi espera al començament del <i>frame</i> per no avançar-se més enllà del nombre de <i>frames</i> que el motor permet tenir en curs.
Semàfors d'imatge disponible	<i>frames</i> en vol	L'adquisició d'imatge els senyala; l'enviament hi espera abans d'escriure a l'adjunt de color.
Semàfors de feina completada	imatges de la cadena	L'enviament els senyala; la presentació hi espera.
Tanques per imatge	imatges de la cadena	Punters a la tanca del <i>frame</i> lògic que va fer servir cada imatge per darrera vegada.

El Codi 4 mostra com s'articulen a l'inici de cada *frame*.

```

// 1. No avançar-se més enllà dels frames en vol permesos.
vkWaitForFences(device, 1, &in_flight_fences[current_frame], true, UINT64_MAX);

// 2. Adquirir la imatge. Retorna un índex ara; el semàfor es senyala
// quan la imatge és realment disponible.
vulkan_swapchain_acquire_next_image_index(
    &ctx, &ctx.swapchain, UINT64_MAX,
    image_available_semaphores[current_frame], 0, &image_index);

// 3. Si un frame anterior encara fa servir aquesta imatge, esperar-lo.
if (images_in_flight[image_index] != 0) {
    vkWaitForFences(device, 1, images_in_flight[image_index], true, UINT64_MAX);
}
images_in_flight[image_index] = &in_flight_fences[current_frame];

```

Codi 4: Sincronització a l'inici del *frame*. Els vectors indexats per *frame* lògic i els indexats per imatge de la cadena d'intercanvi es mantenen separats perquè les seves mides no tenen cap raó de coincidir.

La primera ordenació és que el processador no s'avanci indefinidament al dispositiu. El motor limita el nombre de *frames* lògics en curs simultàniament i, en començar-ne un, espera la tanca corresponent. Les tanques es creen ja senyalitzades perquè els primers *frames* no esperin una senyal que ningú no emetrà.

La segona és que el dibuix no comenci abans que la imatge de destinació estigui realment disponible. L'adquisició retorna un índex immediatament, però la imatge no està llesta fins que el semàfor associat es senyala [5]. L'enviament hi espera a l'etapa d'escriptura de l'adjunt de color, de manera que el processament de vèrtexs pot començar abans i només es retarda l'escriptura.

La tercera és que la presentació no comenci abans que el dibuix hagi acabat, cosa que s'aconsegueix amb el semàfor que l'enviament senyala i que la presentació espera.

5.3.3 Per què els vectors tenen mides diferents

Aquest és el punt no evident del disseny. Els dos nombres es deriven del mateix valor de partida —el mínim d'imatges que la superfície reporta— però no per la mateixa via: el nombre d'imatges el fixa el sistema de presentació, que en pot retornar més de les demanades, mentre que el nombre de *frames* lògics el calcula el motor en crear la cadena d'intercanvi. No hi ha cap raó perquè el resultat coincideixi.

Els objectes lligats al ritme del processador es dimensionen, doncs, pel nombre de *frames* en vol, i els lligats a una imatge concreta es dimensionen pel nombre d'imatges i s'indexen per índex d'imatge. Indexar un semàfor de feina completada pel *frame* lògic en lloc de fer-ho per la imatge faria que la presentació d'una imatge pogués esperar un semàfor que senyalarà l'enviament d'un *frame* diferent. El vector de tanques per imatge tanca el buit restant: abans de reutilitzar una imatge, el motor espera la tanca del *frame* que la va fer servir per darrera vegada.

Aquesta divergència no és hipotètica. Sobre l'equip A el sistema de presentació retorna quatre imatges mentre el motor manté tres *frames* lògics en curs; sobre l'equip B els dos nombres coincideixen a dos. La distinció, per tant, és invisible en un dels dos entorns i determinant a l'altre, la qual cosa il·lustra que només es pot verificar provant sobre maquinari divers.

Convé assenyalar aquí una anomalia detectada en aquest mateix codi. En calcular el nombre d'imatges a sol·licitar, s'afegeix una unitat al mínim que la superfície reporta —pràctica habitual per evitar esperes— i tot seguit una condició la resta de nou, ja que compara el valor amb ell

mateix menys u i resulta sempre certa. La sol·licitud acaba sent, doncs, exactament el mínim. El comentari que acompanya aquesta condició declara la intenció d'igualar el nombre d'imatges i el de *frames* en vol, objectiu que de tota manera no es podria assolir així: el nombre definitiu no el fixa la sol·licitud sinó el sistema de presentació, que en pot retornar més dels demanats, com efectivament fa a l'equip A. L'anomalia no afecta cap de les mesures del capítol 6, que no depenen del nombre d'imatges, i es recull com a treball pendent.

5.3.4 Recursos de *shader* i freqüència d'actualització

La secció anterior tracta la duplicació dels objectes de sincronització. La mateixa qüestió — quantes còpies cal mantenir d'una dada perquè l'amfitrió pugui escriure-la mentre el dispositiu encara llegeix l'anterior— determina també com s'organitzen les dades que els *shaders* consumeixen, i val la pena descriure-ho perquè és on conflueixen dues recomanacions independents que el capítol 7 comenta.

5.3.4.1 El criteri

Un *shader* de materials necessita dades que canvien a ritmes molt diferents. Les matrius de projecció i de vista i els paràmetres de la llum canvien un cop per *frame*; el color difús i la textura canvien quan canvia el material; la matriu de model canvia a cada objecte dibuixat. Tractar-les totes igual és ineficient en un sentit o en l'altre: escriure-les totes a cada dibuix repeteix feina, i escriure-les totes un cop per *frame* és directament incorrecte per a les de l'últim grup.

El criteri adoptat és agrupar-les per freqüència d'actualització i assignar a cada grup el mecanisme de l'API que li correspon. L'especificació ordena els recursos accessibles des d'un *shader* segons el cost d'actualitzar-los, i situa les constants d'inserció com el camí de menor cost per a quantitats petites [5]. La bibliografia d'arquitectura de motors hi arriba pel seu compte, i per raons anteriors a aquesta generació d'API [1].

5.3.4.2 Els tres nivells

El motor en fa tres grups, que la Taula 8 recull.

Taula 8: Recursos del *shader* de materials, agrupats per freqüència d'actualització.

Freqüència	Mecanisme	Contingut	Mida
Per <i>frame</i>	Conjunt 0: búfer uniforme	Projecció, vista, color ambient, direcció i color de la llum, posició de la càmera	256 B
Per material	Conjunt 1: búfer uniforme i mostrejador	Color difús i mapa difús	64 B
Per objecte	Constants d'inserció	Matriu de model	64 B

El repartiment entre el conjunt global i el de material no és només organitzatiu: vincular un conjunt de descriptors invalida els conjunts de numeració superior, de manera que col·locar allò que canvia menys sovint als números baixos redueix el nombre de vinculacions que cal refer [5].

La matriu de model no passa per cap descriptor. S'escriu directament al *command buffer* amb l'ordre d'inserció, immediatament abans de cada dibuix, de manera que no necessita ni reserva de memòria ni conjunt ni actualització.

5.3.4.3 La duplicació, i per què

Cada conjunt de descriptors existeix tantes vegades com imatges té la cadena d'intercanvi, i s'indexa per índex d'imatge. La raó és la mateixa que justifica els vectors de la secció anterior: mentre el dispositiu executa el *frame* anterior encara llegeix el conjunt que el va servir, de manera que escriure-hi el contingut del *frame* següent el corrompria.

Convé assenyalar que la reserva de descriptors es dimensiona pel nombre d'imatges i no pel de *frames* en vol. Són dues xifres que poden divergir —s'ha vist que a l'equip A divergeixen— i dimensionar la reserva per la menor de les dues esgotaria els conjunts disponibles en indexar per imatge.

5.3.4.4 Evitar l'escriptura innecessària

Duplicar els conjunts resol la correcció però no estalvia feina: reescriure'ls a cada *frame* continuaria sent innecessari, ja que el contingut d'un material poques vegades canvia. Aquí és on entra el mecanisme d'identificador i generació descrit al capítol 4. Cada descriptor desa la generació del recurs que hi té vinculat, i l'operació d'actualització només s'emet quan aquesta generació difereix de la del recurs. Un material estable es vincula sense reescriure's mai. És l'ús concret que justifica que el model de recursos mantingui un camp de generació al costat de l'identificador.

5.3.4.5 Costos i una assumpció conservadora

Dos aspectes d'aquest disseny mereixen consignar-se.

El primer és que tant el bloc global com el de material reserven espai que avui no s'utilitza: el global manté seixanta-quatre bytes marcats com a reservats i el de material quaranta-vuit. La raó és que modificar la mida d'aquests blocs obliga a revisar l'alineació de tot el que hi ha a continuació, i mantenir-la fixa fa que afegir-hi un camp no en tingui cap conseqüència.

El segon és una assumpció que el codi fa i que val la pena corregir. El rang de constants d'inserció es declara de cent vint-i-vuit bytes tot i que només se n'escriuen seixanta-quatre, i un comentari al *shader* justifica la xifra dient que és el total garantit. Ho era: cent vint-i-vuit bytes és el mínim que l'especificació garanteix per al nucli de Vulkan, però la versió 1.4 —que és la que el motor sol·licita— l'eleva a dos-cents cinquanta-sis [5]. La declaració actual compromet, doncs, tot el pressupost garantit del nucli per a una dada que n'ocupa la meitat, sobre la base d'un límit més estricte del que la versió emprada imposa. A diferència dels supòsits que descriu la secció següent, aquest és conservador i no provoca cap fallada; però és igualment un valor escrit al codi en lloc de consultat.

5.3.5 Supòsits sobre el dispositiu

La secció anterior descriu el disseny tal com és avui. Arribar-hi va requerir corregir un conjunt de supòsits que el codi feia sobre el dispositiu i que resultaven certs al maquinari de desenvolupament de la implementació de referència però no al d'aquest treball. Els quatre casos, documentats al *commit* a0f9a24, comparteixen la mateixa naturalesa i val la pena enumerar-los perquè il·lustren una lliçó concreta.

Selecció de dispositiu La selecció exigia una GPU discreta com a requisit estricte, de manera que en un portàtil amb només gràfics integrats no trobava cap dispositiu apte i l'arrencada fallava. La correcció fa la selecció en dues passades: la primera manté l'exigència i, si cap dispositiu no la satisfà, la segona la relaxa i accepta qualsevol dispositiu que compleixi la resta de requisits. Les GPU discretes continuen sent preferides quan n'hi ha.

Extensió de la superfície El codi construïa la passada de renderitzat i els *framebuffers* a partir de la mida sol·licitada per a la finestra. La superfície, però, pot imposar una extensió diferent —per escalat fraccionari o per decoracions de finestra—, i les imatges de la cadena d’intercanvi es dimensionen a partir d’aquesta extensió efectiva. Sobre Windows totes dues coincidien; sobre un compositor que n’imposa una altra, no, i els adjunts no coincidien amb la passada. La correcció propaga l’extensió efectiva de tornada perquè tot el dimensionament posterior en derivi.

Recursos per imatge Diversos vectors de recursos indexats per imatge estaven dimensionats amb la constant tres, donant per fet un triple emmagatzematge. El nombre real d’imatges el decideix el controlador i varia entre dispositius. La correcció els dimensiona en temps d’execució a partir del nombre que la cadena reporta, amb un límit superior i una assertió que el verifica.

Filtratge anisotrópic El mostrejador sol·licitava setze mostres de manera fixa, valor que una GPU integrada pot no admetre. La correcció el limita al màxim que el dispositiu declara.

A aquests quatre s’hi afegeixen dos casos més, detectats en portar el motor a l’equip amb GPU dedicada i de naturalesa lleugerament diferent: no són valors presuposats sinó comportament indefinit que una configuració de maquinari concreta posa al descobert.

Punter a un objecte fora d’àmbit En crear la cadena d’intercanvi, el vector amb els índexs de les famílies de cues es declarava dins del bloc condicional que tracta el cas de compartició entre famílies, però l’estructura de creació hi continuava apuntant i només se’n llegia el contingut més avall. La condició que ho activa és que les famílies de gràfics i de presentació siguin diferents, cosa que succeeix a l’equip B —famílies 0 i 2— i no a l’equip A, que en té una de sola. Es tracta de comportament indefinit present a totes dues plataformes; simplement, una no l’exercita.

Estat del subsistema de registre S’hi passava l’adreça del punter en lloc del punter, de manera que l’estat intern del subsistema acabava situat dins de l’estructura de l’aplicació. També és latent en tots dos entorns.

Convé una precisió sobre el primer cas. La fallada que va conduir-hi va deixar de reproduir-se en afegir traces per diagnosticar-la, cosa habitual quan es llegeix memòria ja alliberada, ja que desplaçar la pila canvia el contingut que s’hi troba. No es pot afirmar, doncs, que aquell defecte fos la causa de la fallada observada; sí que era l’únic comportament indefinit del camí afectat i que la condició que l’activa coincideix amb la diferència entre els dos equips.

El denominador comú dels quatre primers és que cadascun d’aquests valors estava escrit al codi en lloc de consultar-se al dispositiu. És, literalment, l’advertència que la documentació de Mantle ja feia el 2015 i que es va citar al capítol 2: no presuposar les propietats del sistema, sinó consultar les que aquest reporta [4]. Trobar-los va exigir entendre què garanteix l’API i què deixa a criteri de la implementació, cosa que no es dedueix de veure funcionar el codi sobre una sola màquina.

5.3.6 Limitacions

L’enregistrament de les ordres es fa en un sol fil d’execució. L’API està dissenyada perquè diversos fils puguin enregistrar *command buffers* en paral·lel, i especifica quins objectes requereixen sincronització externa precisament per permetre-ho [5]; el motor no ho aprofita. És la limitació més rellevant del treball i es discuteix al capítol 7.

L'abast real d'aquesta limitació es pot acotar un cop el motor mesura per separat el temps de l'amfitrió i el del dispositiu, tal com descriu la secció sobre gestió de memòria. Les mesures mostren que el dispositiu completa la feina d'un *frame* en una fracció del temps que dura la iteració, de manera que amb la càrrega gràfica actual no és el dispositiu el que la determina. Convé no llegir-hi més del que diu: el temps restant es reparteix entre treball de l'amfitrió i esperes de sincronització, que el motor no cronometra per separat. El capítol 6 hi torna amb les xifres i la discussió corresponents.

5.4 Sistemes de recursos

La gestió d'actius del motor es reparteix en dos nivells. Un nivell inferior s'ocupa d'obtenir les dades d'un actiu des del disc i lliurar-les en memòria; un nivell superior s'ocupa de convertir-les en recursos utilitzables, de compartir-los i de decidir quan es descarreguen. La separació és recent i, com s'ha indicat, encara no és completa.

5.4.1 El sistema de recursos

El nivell inferior és un registre de carregadors especialitzats. Cada carregador declara el tipus de recurs que sap tractar, el subdirectori on aquest tipus resideix, i una parella de funcions per carregar-lo i descarregar-lo. Qui necessita un actiu no obre cap fitxer: demana un nom i un tipus, i el sistema localitza el carregador corresponent, en compon la ruta a partir d'una arrel configurable i li delega la feina.

El resultat s'entrega en una estructura uniforme que identifica el carregador que l'ha produït, el nom i la ruta completa de l'actiu, i un bloc de dades amb la seva mida. El camp que identifica el carregador és el que permet que la descàrrega sigui simètrica sense que qui la demana hagi de recordar de quin tipus era el recurs.

Hi ha quatre carregadors implementats —text, binari, imatge i material— sobre un conjunt de tipus que en preveu dos més, malla estàtica i tipus definit per l'usuari, encara sense implementació.

Aquesta capa és la que va permetre retirar l'accés directe al sistema de fitxers que els sistemes de textures i de materials feien abans, i amb ell les rutes compostes amb literals dins del codi. És, en la seva funció de localització i càrrega, la capa unificada d'accés a actius que el capítol 4 assenyalava com a mancança.

5.4.2 Els sistemes consumidors

Sobre aquesta base hi ha tres sistemes —textures, materials i geometries— que comparteixen la mateixa estructura: un vector de registres, una taula de dispersió que associa noms amb referències, i la política de cicle de vida descrita al capítol 4, basada en recompte de referències i en un indicador d'alliberament automàtic.

Tots tres mantenen, a més, un recurs per defecte que no prové de disc. La textura per defecte és un tauler d'escacs generat per codi, i existeix perquè el renderitzador pugui funcionar sense cap actiu disponible i perquè un actiu que no es pugui carregar tingui un substitut visible en lloc de provocar una fallada.

5.4.3 Un actiu, de punta a punta

El camí complet d'un material il·lustra com encaixen les dues capes. La Figura 4 en resumeix els passos.

Petició	Es demana un material pel seu nom.
Localització	El sistema de recursos resol la ruta i tria el carregador de materials.
Anàlisi	El carregador llegeix el fitxer de text i n'omple una estructura de configuració amb el nom, el color difús i el nom del mapa difús.
Adquisició de textura	La configuració nomena una textura; el sistema de materials l'adquireix del de textures, que al seu torn torna a passar pel sistema de recursos, aquest cop amb el carregador d'imatges.
Recursos de dispositiu	El renderitzador reserva el conjunt de descriptors del material i en desa l'identificador intern.
Ús	En dibuixar, el material aporta el color difús i la textura al conjunt de descriptors per objecte.

Figura 4: Camí de càrrega d'un material, des del fitxer fins al dibuix.

El pas d'adquisició de textura és el que fa que el recompte de referències tingui sentit: el material no conté la textura sinó que la nomena, de manera que dos materials que anomenin la mateixa textura en comparteixen una sola còpia a la memòria del dispositiu.

El fitxer de material, que el Codi 5 reproduceix, és el primer actiu del motor dirigit per dades en el sentit que li dona la bibliografia: una definició que resideix en un fitxer de text i no al codi.

```
#material file

version=0.1
name=test_material
diffuse_color=1.0 1.0 1.0 1.0
diffuse_map_name=cobblestone_floor_tiled_32
```

Codi 5: Fitxer de definició d'un material. El material anomena la seva textura en lloc de contenir-la, cosa que permet que diversos materials en comparteixin una sola còpia a la memòria del dispositiu.

Inclou un camp de versió que actualment no s'interpreta, previst per permetre'n l'evolució del format.

5.4.4 Unificació parcial

Convé ser precís sobre l'abast d'aquesta reorganització. El sistema de recursos unifica la localització i la càrrega, però no la gestió del cicle de vida: textures, materials i geometries continuen mantenint cadascun la seva taula de referències i la seva política d'alliberament. La duplicació que el capítol 4 assenyalava entre dos subsistemes afecta ara tres.

La unificació completa exigiria que el recompte de referències i la identificació per nom residissin també al nivell inferior, de manera que els sistemes especialitzats només aportessin la interpretació específica de cada tipus. Es recull com a línia de continuació al capítol 7.

5.5 Resta de subsistemes

Les seccions anteriors tracten en profunditat els quatre subsistemes amb més càrrega de decisió. Aquesta recull l'inventari complet del motor, per situar-los dins del conjunt.

El codi font del motor consta de 84 fitxers i prop de 12.900 línies, de les quals 10.779 no són buides, sense comptar-hi la biblioteca de tercers emprada per descodificar imatges. La Taula 9 els agrupa segons les capes del model presentat al capítol 2.

La darrera columna indica on trobar-ne el detall. Els subsistemes que no tenen una secció pròpia en aquest capítol es documenten a les capçaleres del seu codi font, comentades una per una, i al corpus de documentació tècnica descrit a la secció sobre el mètode de treball. S'ha

preferit aquesta remissió a reproduir aquí la interfície pública de cada mòdul, que duplicaria sense afegir-hi res el que el codi ja declara.

Taula 9: Inventari de subsistemes del motor.

Subsistema	Responsabilitat	Detall
<i>Capa d'independència de plataforma</i>		
Plataforma	Finestra, drenatge d'esdeveniments del sistema, rellotge absolut, reserva de memòria i sortida per consola. Implementada per a Windows i per a Linux sobre XCB, X11 i xcbcommon.	§5.1
Sistema de fitxers	Lectura i escriptura de fitxers en mode text i binari.	Repositori
<i>Sistemes bàsics</i>		
Registre i assercions	Sis nivells de severitat, amb els inferiors eliminats en compilacions de producció.	Repositori
Memòria	Reserva etiquetada per categoria, amb totals acumulats per etiqueta i recompte d'assignacions.	§5.2
Assignador lineal	Repartiment incremental sobre un bloc fix, sense alliberament individual.	§5.2
Instrumentació	Mesura del cost de cada fase del <i>frame</i> sobre una finestra mòbil, amb acumuladors independents per a operacions fora del bucle.	§5.2
Esdeveniments	Publicació i subscripció per codi d'esdeveniment.	Repositori
Entrada	Estat de teclat i ratolí, amb comparació entre l'estat actual i el previ per detectar transicions.	Repositori
Rellotge	Mesura de temps transcorregut, base del càlcul del pas de temps [23].	Repositori
Cadenes	Manipulació de cadenes i conversió a tipus numèrics i vectorials.	Repositori
Contenidors	Vector dinàmic amb capçalera de metadades i taula de dispersió.	Repositori
Matemàtiques	Vectors, matrius, quaternions i les projeccions i transformacions que el renderitzador necessita.	Repositori
<i>Gestió de recursos</i>		
Sistema de recursos	Registre de carregadors especialitzats i resolució de rutes a partir d'una arrel configurable.	§5.4
Carregadors	Quatre implementacions especialitzades: text, binari, imatge i material.	§5.4
Textures	Adquisició per nom amb recompte de referències i textura per defecte generada per codi.	§5.4
Materials	Adquisició per nom a partir de fitxers de configuració, amb color difús i mapa de textura.	§5.4

Subsistema	Responsabilitat	Detall
Geometria	Registre de geometries i generadors de plans i de cubs amb normals.	§5.4
<i>Renderitzador</i>		
Part independent de l'API	Estat de projecció, vista i il·luminació, i despatx cap a la implementació activa mitjançant una taula de punters a funció.	§5.1
Implementació de Vulkan	Instància, dispositiu, cadena d'intercanvi, passades de renderitzat, <i>command buffers</i> , canonades, búfers, imatges i sincronització.	§5.3
<i>Shaders</i> integrats	Dos programes: el de materials, amb il·luminació direccional, i el d'interfície d'usuari.	§5.3

5.5.1 Evolució de l'estructura

L'inventari actual no coincideix amb el que hauria resultat d'anar afegint subsistemes sense revisar-ne cap. Dos casos il·lustren com ha canviat l'estructura durant el desenvolupament.

El primer és la incorporació del suport per a múltiples passades de renderitzat. Fins aleshores el motor tenia una única passada i mòduls independents per als *framebuffers* i per a les tanques de sincronització. En generalitzar el tractament de passades, tots dos van deixar de tenir entitat pròpia i van passar a formar part de l'abstracció de passada, de manera que els fitxers corresponents es van eliminar. El nombre de mòduls va disminuir mentre la funcionalitat augmentava.

El segon és l'aparició del sistema de recursos. Els sistemes de textures i de materials havien nascut accedint directament al sistema de fitxers i component les rutes amb literals dins del codi. La introducció d'una capa de càrrega amb carregadors especialitzats va permetre retirar aquest accés directe i centralitzar la resolució de rutes. Es tracta, precisament, del tipus de capa genèrica que la secció sobre el model de recursos del capítol 4 assenyalava com a mancança, tot i que la unificació encara no és completa.

6 Resultats i discussió

6.1 Metodologia de mesura

6.1.1 Instrument

Totes les mesures d'aquest capítol provenen de la capa d'instrumentació pròpia descrita al capítol 5, sense recórrer a eines externes. Aquesta capa proporciona tres magnituds de naturalesa diferent, i convé no confondre-les:

Temps d'amfitrió per fase Cronometrat amb el rellotge monotònic del sistema, acumulat dins de cada *frame* i reportat sobre una finestra mòbil de cent vint mostres. Mesura el que el processador dedica a cada fase.

Temps de dispositiu Obtingut amb consultes de marca de temps escrites al flux d'ordres. Mesura el que la GPU dedica a executar el *frame*. Sobre el maquinari emprat, un tic de marca equival a 52,08 nanosegons.

Acumuladors de vida sencera Temps total i nombre d'execucions per secció des de l'arrencada, independents del bucle de *frames*. Són els que permeten mesurar operacions que només succeeixen una vegada, com les càrregues de recursos.

L'informe s'emet un cop per segon al registre, de manera que cada execució mesurada es captura redirigint la sortida a un fitxer.

6.1.2 Protocol

Les configuracions que es comparen se seleccionen en temps d'execució mitjançant variables d'entorn, de manera que totes les mesures provenen del mateix binari compilat una sola vegada. Això elimina qualsevol diferència atribuïble a la compilació.

Cada configuració s'executa tres vegades sobre cadascun dels dos equips de la Taula 5. Les execucions duren uns set segons, prou perquè la finestra de mostres s'ompli diverses vegades i perquè les operacions d'arrencada s'hagin completat.

6.1.3 Limitacions de l'instrument

Tres limitacions condicionen què es pot afirmar a partir d'aquestes mesures, i val la pena enunciar-les abans dels resultats.

La primera afecta el temps de dispositiu. Una consulta només es pot llegir quan la feina que la va escriure ha acabat, de manera que la xifra obtinguda en un *frame* correspon al que es va enviar tantes iteracions enrere com *frames* en vol permeti el motor. Sobre una finestra de cent vint mostres el desfasament no altera les mitjanes, però invalida qualsevol lectura de mesures individuals.

La segona afecta el temps per *frame*. La capa no descarta cap període d'escalfament: la finestra comença a omplir-se amb el primer *frame* i llisca, de manera que l'informe que es consulta reflecteix els cent vint *frames* immediatament anteriors, siguin quins siguin. Com es veurà, això produeix una variància entre execucions prou gran com perquè les mesures per *frame* no permetin distingir configuracions que difereixen poc.

La tercera afecta les xifres d'ocupació de memòria. El comptador s'inicialitza després que s'hagin reservat l'estat de l'aplicació i el bloc de seixanta-quatre megabytes, i durant bona part del desenvolupament cap dels dos no hi apareixia. La correcció documentada a §5.2 declara totes dues reserves un cop el comptador és viu, de manera que ara l'informe recull tot el que el

motor demana al sistema operatiu. El que continua sense mesurar és quina part del bloc s'ocupa efectivament i què reserva el controlador de l'API gràfica pel seu compte; per aquesta raó les xifres d'ocupació es donen com a descripció del model de memòria al capítol 5 i no s'utilitzen com a resultat experimental en aquest capítol.

6.2 Assoliment dels objectius funcionals

Aquesta secció respon un per un als objectius específics plantejats al capítol 1, amb l'evidència corresponent i l'estat en què ha quedat cadascun. La Taula 10 en resumeix el balanç.

Taula 10: Balanç dels objectius específics.

Objectiu	Estat	Evidència
Capa de plataforma portable	Assolit	§5.1, §5.5
Renderitzador amb recursos des de disc i il·luminació	Assolit	§5.3, §5.4, §5.5
Model de gestió de memòria explícit	Assolit amb limitacions	§5.2
Validació sobre les dues arquitectures de memòria	Assolit	§5.3, §6.3
Documentació de les decisions contra fonts primàries	Assolit	Cap. 4, Annex A
Capa d'instrumentació pròpia	Assolit	§5.2, §6.1
Avaluació empírica de decisions del renderitzador	Parcial	§6.3

6.2.1 Capa de plataforma

El motor defineix una interfície de plataforma implementada dues vegades, per a Windows i per a Linux sobre XCB, X11 i xcbcommon, i seleccionada en temps de compilació. Cap subsistema per sobre d'aquesta capa conté codi condicional per sistema operatiu.

L'evidència més sòlida d'aquesta independència no és estructural sinó històrica: el projecte es va desenvolupar inicialment sobre Windows i posteriorment es va portar a Linux, que ha estat l'entorn principal des d'aleshores, tal com documenta el capítol 3. El canvi va requerir escriure la implementació de la capa i resoldre incompatibilitats de compilació, però no modificar la lògica dels subsistemes superiors.

Aquesta independència s'ha tornat a verificar en portar a Windows les ampliacions desenvolupades sobre Linux. El codi compila sense cap modificació específica de plataforma, i els tres riscos que s'havien anticipat —el canvi al format de vèrtex, l'ús d'una funció de la biblioteca estàndard per llegir variables d'entorn i els subsistemes de nova incorporació— no van arribar a materialitzar-se.

La Figura 5 mostra el motor executant-se sobre l'altra plataforma.



Figura 5: El motor sobre Windows i GPU dedicada, amb la mateixa escena que s'analitza tot seguit: el cub texturat i il·luminat i el quadrilàter d'interfície. El resultat és equivalent en tots dos entorns tot i que el controlador, el sistema de finestres i el repartiment de famílies de cues del dispositiu són diferents.

L'execució, en canvi, va requerir corregir dos defectes que el capítol 5 documenta. Convé destacar-ne la naturalesa: cap dels dos és específic de Windows. Tots dos són comportament indefinit present també a l'entorn Linux, on no es manifesta perquè el maquinari no exercita la condició que els activa. El port, doncs, no va exposar una mancança de la capa de plataforma sinó dos defectes latents del motor, la qual cosa reforça l'argument d'aquest objectiu en lloc de matisar-lo.

6.2.2 Renderitzador

El renderitzador carrega geometria, textures i materials des de disc mitjançant el sistema de recursos descrit a §5.4, els il·lumina amb un model de reflexió difusa i especular amb una llum direccional, i els dibuixa amb una càmera que l'usuari pot desplaçar i orientar. Disposa de dues passades de renderitzat, una per al món i una per a la interfície d'usuari.

La Figura 6 mostra el motor en execució i permet comprovar-ho.

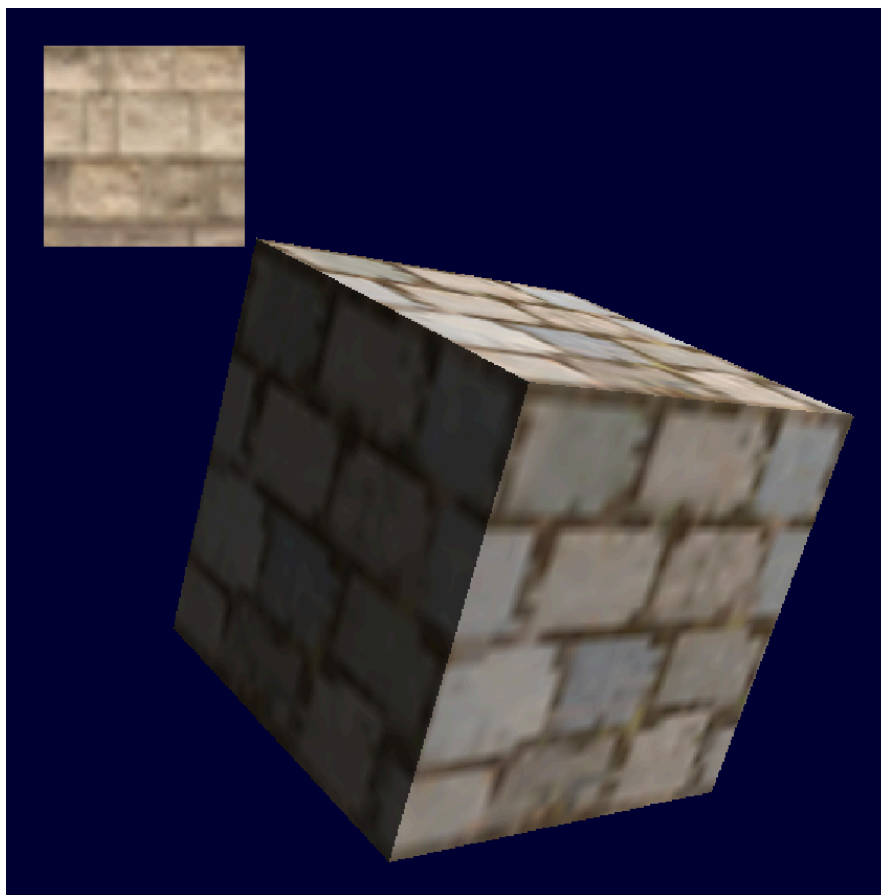


Figura 6: El motor en execució. El cub el genera el sistema de geometria amb una normal per cara; la textura i el color difús provenen d'un fitxer de material carregat des de disc. Les tres cares visibles reben il·luminacions diferents segons l'angle que formen amb la llum direccional: la dreta hi està encarada, la superior hi queda obliqua i l'esquerra n'està girada i només rep el terme ambient. El quadrilàter del cantó superior esquerre es dibuixa a la passada d'interfície, amb un material i una projecció diferents dels del món.

Aquesta única imatge evidencia la major part de l'objectiu: la geometria prové del generador descrit a §5.4, la textura i el color del material provenen d'un fitxer de configuració llegit des de disc, la variació de lluminositat entre cares demostra que el terme difús opera sobre les normals per cara, i la presència simultània del cub i de l'element d'interfície evidencia les dues passades de renderitzat. El desplaçament i l'orientació de la càmera no es poden mostrar en una imatge fixa, però són el mecanisme amb què es va obtenir aquest enquadrament.

El terme especular, en canvi, no s'il·lustra per separat, i convé explicar per què. En una superfície plana la normal és constant, de manera que el factor especular ho és també i el terme es reparteix de manera uniforme per cara en lloc de concentrar-se en un reflex localitzat. Il·lustrar-lo exigiria geometria corba o variació de normal per píxel, cap de les quals forma part de l'abast d'aquest treball. El terme hi contribueix —forma part del càlcul que fa el *shader* de materials— però la geometria disponible no permet aïllar-lo visualment.

6.2.3 Model de gestió de memòria

L'estat dels onze subsistemes resideix en un únic bloc contigu reservat a l'arrencada i repartit mitjançant el patró d'inicialització en dues crides. Cap subsistema decideix on resideix el seu

estat ni quan s'allibera: totes dues coses són decisions de qui el condueix. L'objectiu, formulat com a control sobre la ubicació i el cicle de vida, s'ha assolit.

Es consigna com a assolit amb limitacions per dues raons documentades a §5.2. La primera és que cap dels dos assignadors alinea les reserves. La segona és que l'assignador lineal no comprova el punter nul que retorna en exhaurir-se abans de lliurar-lo al subsistema, de manera que esgotar el bloc produiria un accés a memòria nul·la en lloc d'un diagnòstic. Cap de les dues invalida el model, però totes dues afecten la qualitat de la seva implementació. El punt cec del comptador d'ocupació, que hi figurava com a tercera limitació, s'ha corregit en el curs de la redacció d'aquesta memòria.

6.2.4 Validació sobre les dues arquitectures de memòria

Aquest objectiu s'ha assolit. El motor s'executa correctament sobre els dos equips descrits al capítol 3, i l'adaptació que ho va fer possible —documentada a §5.3— constitueix una de les aportacions pròpies del treball. En aquest sentit, la validació funcional és completa.

El contrast del comportament també s'ha completat: l'experiment de §6.3 s'ha executat sobre les dues màquines, i el motor hi selecciona l'estratègia consultant el dispositiu en lloc de deduir-la de la seva categoria. El resultat matisa la premissa de partida, ja que la GPU dedicada també exposa memòria pròpia visible des de l'amfitrió, si bé acotada. El que queda fora d'abast és generalitzar: dues màquines permeten contrastar dues arquitectures, no caracteritzar-les.

6.2.5 Documentació de les decisions

El capítol 4 recull sis decisions estructurals, cadascuna amb el problema que planteja, les alternatives considerades, el criteri aplicat i el cost assumit, i cadascuna referida a la font primària que la fonamenta. L'Annex A en conté la correspondència detallada, i el corpus de documentació tècnica del repositori —descriu a §3.1— l'acompanya al nivell de cada subsistema.

Aquest objectiu inclou un criteri que convé destacar: que la justificació sigui possible de manera independent. El treball ha generat dos casos en què l'estudi de les fonts va modificar el que s'hauria escrit sense elles. El primer és l'argument sobre el llenguatge de §4.2, on la font que es pretenia citar afirma el contrari del que se li volia atribuir. El segon és l'experiment de §6.3, on una pràctica establerta resulta prescindible sobre el maquinari emprat.

6.2.6 Capa d'instrumentació

El motor mesura el cost de cada fase del *frame* al processador, el temps que el dispositiu dedica a executar-lo mitjançant consultes de marca de temps, i el cost acumulat d'operacions que succeeixen fora del bucle. No s'ha emprat cap eina externa de perfilatge en tot el capítol 6.

Queda fora de l'objectiu, i per tant no se'n consigna com a mancança, la representació d'aquestes dades en pantalla, que hauria estat útil durant el desenvolupament però que no era necessària per a l'avaluació.

6.2.7 Avaluació empírica

Assolit parcialment. Dels quatre experiments plantejats se n'ha completat un, el de les estratègies de transferència, amb tres repeticions per configuració i un control que permet atribuir-ne el resultat. Els tres restants es basen en mesures per *frame* i requereixen resoldre prèviament la variància descrita a §6.1.

6.3 Experiments

6.3.1 Què es vol mesurar i per què

El capítol 4 enuncia sis decisions estructurals i, per a cadascuna, el criteri que la fonamenta i el cost que se li atribueix. Aquests criteris i aquests costos provenen de les fonts o del raonament, no de l'observació: fins aquest punt de la memòria, cap d'ells no s'ha comprovat sobre el motor construït. L'avaluació neix precisament d'aquí, i el seu propòsit és sotmetre a mesura les afirmacions que fins ara es donaven per bones.

Això delimita què té sentit mesurar. No es busca situar el motor respecte de cap altre —l'abast n'és massa desigual perquè la comparació signifiqui res— ni caracteritzar el maquinari, que és feina d'eines especialitzades. El que es busca és, per a cada decisió documentada, una magnitud que permeti dir si el que se n'esperava succeeix.

Sota aquest criteri es van plantejar quatre experiments, cadascun lligat a una decisió concreta:

1. **Estratègia de transferència de dades cap al dispositiu.** Comprovar si la còpia intermèdia que la bibliografia prescriu és necessària sobre les dues arquitectures de memòria disponibles, o si el motor pot evitar-la consultant els tipus de memòria que el dispositiu exposa. Es mesura amb acumuladors de vida sencera, perquè les càrregues de recursos succeeixen una sola vegada.
2. **Repartiment del temps d'iteració entre fases.** Determinar quina fase del bucle domina el temps de *frame* i quina part correspon al dispositiu, per contrastar la valoració que el capítol 4 fa de l'enregistrament en un sol fil.
3. **Nombre de *frames* en vol.** Mesurar l'efecte del grau de solapament entre amfitrió i dispositiu sobre el temps d'iteració, que és el paràmetre que governa el model de sincronització descrit a §5.3.
4. **Ordenació dels recursos de *shader* per freqüència d'actualització.** Verificar que l'ordenació que el capítol 4 adopta per recomanació de les dues fonts té l'efecte que se li atribueix sobre el cost d'actualització.

Només el primer s'ha completat, i és l'únic que aquesta secció reporta. La raó és la limitació documentada a §6.1: els tres restants depenen de mesures per *frame*, i la variància entre execucions que l'absència d'un període d'escalfament introdueix supera les diferències que caldria distingir. El primer és, no per casualitat, l'únic dels quatre que es recolza en acumuladors de vida sencera i que, per tant, aquella limitació no afecta.

Presentar-ne un de quatre és una mancança de l'avaluació i es consigna com a tal al capítol 7. Val la pena assenyalar, però, que el que s'ha completat no és el més fàcil dels quatre sinó el que contradiu una pràctica establerta, i que el que bloqueja els altres tres és un defecte conegut de l'instrument amb una solució també coneguda.

6.3.2 Búfer intermedi enfront d'escriptura directa

6.3.2.1 Hipòtesi

El patró de càrrega de recursos cap a memòria de dispositiu que la bibliografia descriu —reservar un búfer visible des de l'amfitrió, copiar-hi les dades, ordenar una còpia cap a un búfer local al dispositiu i destruir l'intermedi— parteix del supòsit que la memòria local al dispositiu no és accessible des del processador. Sobre una arquitectura de memòria unificada aquest supòsit pot no complir-se, cas en què la còpia intermèdia seria sobrecost prescindible.

6.3.2.2 Fonament

Abans de mesurar res cal comprovar si el supòsit es compleix a cada maquinari, i el resultat d'aquesta comprovació és ja el primer resultat de l'experiment.

El dispositiu de l'equip A exposa un únic munt d'11,48 GiB marcat com a local al dispositiu, i set tipus de memòria dels quals dos són alhora locals al dispositiu i visibles des de l'amfitrió. Es tracta, literalment, del cas que l'especificació preveu quan adverteix que en algunes arquitectures pot haver-hi un sol munt utilitzable per a qualsevol propòsit [5]. La còpia intermèdia, en aquest maquinari, copia memòria cap a una regió de la mateixa naturalesa.

L'equip B, amb GPU dedicada, contradiu la previsió de partida. Exposar tres munts: un de 5,83 GiB local al dispositiu, un de 7,71 GiB sense banderes, i un tercer de només 214 MiB que és alhora local al dispositiu i accessible des de l'amfitrió. Aquest tercer munt correspon a la finestra que el dispositiu mapa a l'espai d'adreces del processador, i fa que també en una arquitectura de memòria separada existeixi un tipus de memòria que satisfà les condicions de l'escriptura directa: el tipus 5, únic dels sis que reuneix les tres propietats requerides.

La conseqüència és que la disponibilitat d'escriptura directa no distingeix les dues arquitectures, com s'havia previst. El que les distingeix és la capacitat, il·limitada en la pràctica a l'equip A i acotada a 214 MiB a l'equip B. Els búfers de geometria del motor en sumen 36, de manera que hi caben, però el marge no és ampli: afegir tangents al format de vèrtex portaria el búfer de vèrtexs sol a 48 MiB.

6.3.2.3 Muntatge

S'ha afegit al motor un camí de càrrega alternatiu que, quan el búfer de destinació resideix en memòria visible des de l'amfitrió, hi escriu directament i evita tant la reserva intermèdia com l'ordre de còpia. L'elecció entre les dues estratègies es fa en temps d'execució. El motor no dona per fet que la memòria adequada existeixi: consulta els tipus disponibles i, si no en troba cap que sigui alhora local al dispositiu i visible des de l'amfitrió, retorna a la via del búfer intermedi i ho registra.

Com a control s'utilitzen les càrregues de textures. Les imatges amb disposició òptima no es poden escriure directament, de manera que aquest camí continua emprant el búfer intermedi en totes dues configuracions. Qualsevol variació que hi aparegui és, per tant, atribuïble a la variabilitat entre execucions i no a l'estratègia.

6.3.2.4 Resultats

Cada execució efectua vuit càrregues de búfer i tres de textura. La Taula 11 recull les tres repeticions de cada configuració sobre cadascun dels dos equips.

Taula 11: Temps de càrrega de recursos, en mil·lisegons, segons l'estratègia de transferència i l'equip.

Estratègia	Rep. 1	Rep. 2	Rep. 3	Mitjana	Factor
<i>Equip A — càrrega de búfers de geometria</i>					
Búfer intermedi	12,281	12,454	4,073	9,603	
Escriptura directa	0,077	0,074	0,061	0,071	136×
<i>Equip B — càrrega de búfers de geometria</i>					
Búfer intermedi	5,524	6,404	5,554	5,827	

Estratègia	Rep. 1	Rep. 2	Rep. 3	Mitjana	Factor
Escriptura directa	0,021	0,022	0,019	0,021	282×
<i>Equip A — càrrega de textures (control)</i>					
Amb intermedi	13,789	14,126	13,350	13,755	
Amb directa	13,369	12,026	14,817	13,404	
<i>Equip B — càrrega de textures (control)</i>					
Amb intermedi	5,053	5,502	5,113	5,223	
Amb directa	4,827	5,486	5,599	5,304	

En cap dels dos equips el motor no va haver de recórrer al búfer intermedi: tots dos disposen del tipus de memòria requerit, i en el cas de l'equip B es va verificar que el búfer de geometria hi acaba efectivament assignat —el filtre de tipus admissibles que retorna el dispositiu deixa el tipus 5 com a únic candidat que satisfà les tres propietats.

6.3.2.5 Lectura

L'escriptura directa redueix el temps de càrrega de geometria en tots dos entorns: un factor de cent trenta-sis a l'equip A i de dos-cents vuitanta-dos a l'equip B. El control es manté estable en tots dos casos —una diferència del dos i mig i de l'u i mig per cent respectivament, inferior a la dispersió del mateix control—, cosa que permet atribuir la millora a l'estratègia i no a la variabilitat entre execucions.

La dispersió mereix una observació. A l'equip A la via amb búfer intermedi oscil·la entre 4,07 i 12,45 mil·lisegons, un factor de tres entre el mínim i el màxim; l'escriptura directa es manté entre 0,061 i 0,077, dins d'un marge del vint-i-cinc per cent. A l'equip B la diferència de predictibilitat és menys marcada però va en el mateix sentit. La via directa no és només més ràpida sinó més estable, cosa coherent amb el fet que elimina una reserva de memòria, una ordre de còpia i una espera de cua, tres operacions el cost de les quals depèn de l'estat del sistema.

6.3.2.6 Què mesuren aquestes xifres

Les dues branques comparades no executen la mateixa feina, i convé ser explícit sobre què significa la diferència abans d'interpretar-la.

La via del búfer intermedi reserva memòria, hi copia les dades, enregistra i envia una ordre de còpia i n'espera la finalització. La via directa es redueix a una còpia de memòria. El que la taula compara és, per tant, el cost que cada estratègia imposa a l'amfitrió, no el temps que les dades triguen a ser utilitzables pel dispositiu.

Aquesta distinció és especialment rellevant a l'equip B. Les escriptures a la finestra accessible des de l'amfitrió es publiquen sobre el bus del dispositiu sense que el processador n'espera la finalització, de manera que els vint-i-un microsegons mesurats corresponen al cost d'emetre-les i no al de completar-les. La xifra és vàlida com a comparació de cost d'estratègia i no ho és com a mesura d'amplada de banda ni de latència de transferència.

Les mesures de l'equip B es van prendre sobre un binari que duia instrumentació de diagnòstic addicional, retirada posteriorment. Una execució de comprovació amb el codi net dona 5,268 mil·lisegons per a la via del búfer intermedi, dins del rang de les tres repeticions, de manera que l'instrument no distorsionava la mesura.

Les sis execucions de cada equip es conserven senceres a l'Annex C, de manera que qualsevol xifra d'aquesta taula és comprovable contra el registre que la va produir.

6.3.2.7 Abast del resultat

Convé delimitar què s'ha demostrat. Les càrregues mesurades succeeixen a l'arrencada, de manera que l'estalvi absolut és d'uns deu mil·lisegons una sola vegada en tota l'execució, magnitud irrellevant per a l'experiència d'ús. Les mesures per *frame* no mostren cap diferència atribuïble a la configuració, cosa esperada atès que no hi ha càrregues durant el bucle; la variància descrita a la secció de metodologia és, a més, prou gran com perquè aquestes mesures no permetessin distingir-les encara que n'hi hagués.

El valor del resultat no és, doncs, l'estalvi de temps, sinó el que estableix sobre la decisió de partida. Es tracta amb detall a la secció següent.

6.4 Discussió

6.4.1 La condició no és la que sembla

La previsió de partida d'aquest experiment era que l'escriptura directa seria possible sobre l'equip de memòria unificada i impossible sobre el de memòria dedicada, i que la comparació entre tots dos establiria que la necessitat del búfer intermedi és una propietat del maquinari. La previsió era equivocada, i la manera com ho és resulta més instructiva que si s'hagués complert.

Tots dos equips exposen memòria alhora local al dispositiu i visible des de l'amfitrió. La diferència no és la disponibilitat sinó la naturalesa i la capacitat: a l'equip A perquè tota la memòria ho és, en tractar-se d'una arquitectura unificada; a l'equip B perquè el dispositiu mapa una finestra de 214 MiB de la seva memòria dedicada a l'espai d'adreces del processador.

La conseqüència pràctica és que la comprovació que el motor efectua —si existeix un tipus de memòria amb les tres propietats requerides— resulta insuficient com a criteri de decisió. És condició necessària però no suficient: no distingeix una arquitectura on tota la memòria és accessible d'una on només ho és una finestra reduïda, ni informa del cost relatiu d'escriure-hi enfront de transferir-hi mitjançant el motor de còpia del dispositiu.

6.4.2 Què queda establert i què no

Les mesures estableixen que el cost que la via del búfer intermedi imposa a l'amfitrió és substancial i evitable quan existeix el tipus de memòria adequat, i que aquest cost és a més poc previsible. Això val per a totes dues arquitectures.

No estableixen, en canvi, que l'escriptura directa sigui preferible en general. Com s'ha exposat a la secció anterior, les dues branques no mesuren la mateixa feina i les escriptures a la finestra de l'equip B es publiquen sense esperar-ne la finalització. Per a volums de dades més grans, o quan interressi el moment en què les dades són efectivament utilitzables pel dispositiu i no el cost d'emetre l'escriptura, la comparació podria invertir-se: el motor de còpia del dispositiu està dissenyat per moure dades de manera més eficient que una successió d'escriptures del processador sobre el bus.

L'afirmació que el treball sosté és, per tant, més acotada que la que es pretenia demostrar i, alhora, més aplicable: el patró del búfer intermedi no és incondicionalment necessari, la condició que en determina la necessitat és consultable al dispositiu, i la consulta que cal fer és més fina que la simple existència d'un tipus de memòria.

6.4.3 L'exemple de la finestra acotada

L'equip B il·lustra per què aquesta finor importa. La finestra accessible des de l'amfitrió és de 214 MiB i els búfers de geometria del motor n'ocupen 36. La via directa hi funciona avui, però el marge és limitat: ampliar el format de vèrtex amb tangents portaria el búfer de vèrtexs sol a 48 MiB, i qualsevol escena amb geometria abundant esgotaria el munt. Un motor que decidís l'estratègia únicament a partir de l'existència del tipus de memòria acabaria fallant sobre aquest maquinari en créixer l'escena, sense que res en el criteri emprat n'hagués advertit.

6.4.4 La coherència amb el marc del capítol 2

Aquest resultat tanca un argument que travessa la memòria sencera, i val la pena fer-lo explícit.

El capítol 2 documentava que el trasllat de responsabilitat cap a l'aplicació és el principi fonamental d'aquesta generació d'API, i citava l'avertència que la documentació de Mantle ja feia el 2015: l'aplicació no ha de presuposar la visibilitat de la memòria sinó consultar les propietats que el sistema reporta [4]. L'especificació de Vulkan recull el mateix i preveu explícitament el cas d'una arquitectura amb un únic munt utilitzable per a qualsevol propòsit [5].

El capítol 5 documenta sis defectes del motor que només es manifesten sobre maquinari diferent del de desenvolupament: quatre supòsits sobre propietats del dispositiu escrites al codi en lloc de consultades, i dos casos de comportament indefinit que una configuració concreta de famílies de cues posa al descobert. Aquest capítol n'afegeix un setè, de naturalesa diferent dels anteriors: el motor no fallava en cap dels dos equips, simplement hi dedicava temps innecessari.

Val la pena aturar-se en com es va detectar cadascun, perquè el conjunt dibuixa tres vies independents i cap no cobreix el que cobreixen les altres. Els sis primers es van fer visibles executant el motor sobre maquinari que no era el de desenvolupament: no arrencava o es tancava de manera anòmla. El setè no es podia detectar així, perquè no produïa cap símptoma; només apareix en mesurar.

Hi ha una tercera via, i el capítol 5 en recull el cas. La inicialització del sistema d'esdeveniments netejava vuit bytes d'una taula de cent vint-i-vuit quilobytes, i no havia fallat mai perquè l'assignador lliura la memòria ja neta i no en reutilitza cap regió. Ni executar el motor en un altre equip ni mesurar l'haurien revelat: una propietat d'un altre component el cobria completament. Va aparèixer en llegir el codi per descriure'l en aquesta memòria, i el fet que funcionés no el feia menys defecte: un programa que és correcte per accident no es distingeix d'un de correcte fins que l'accident canvia.

El balanç metodològic és, doncs, que les tres vies són complementàries: executar sobre maquinari divers detecta els supòsits sobre el dispositiu, mesurar detecta el treball innecessari que no falla, i llegir el codi detecta el que està cobert per un accident d'implementació.

6.4.5 L'asimetria entre amfitrió i dispositiu

La instrumentació de temps de dispositiu aporta una segona observació. A les sis execucions registrades, el temps que la GPU dedica a executar el treball d'una iteració no supera en cap cas el setze per cent del temps total d'aquesta iteració, i en la majoria se situa al voltant del cinc per cent.

Convé ser precís sobre què significa aquesta xifra, perquè admet una lectura excessiva. El temps d'iteració no és temps de processament de l'amfitrió: inclou també l'espera de la tanca del *frame* lògic i l'adquisició de la imatge de presentació, operacions que bloquegen sense consumir processador. Afirmar a partir d'aquestes dades que el motor està limitat pel camí d'enviament seria, doncs, anar més enllà del que mesuren.

El que sí que estableixen és que el dispositiu no és el factor limitant. Amb la càrrega gràfica actual —un cub texturat i un quadrilàter d'interfície— la GPU completa la feina molt abans que la iteració acabi, i el temps restant es reparteix entre treball de l'amfitrió i espera. Separar aquestes dues components requeriria instrumentar per separat les esperes de sincronització, i es recull com a treball pendent.

Aquesta observació matisa la valoració de la limitació més rellevant del motor. Que el renderitzador enregistri en un sol fil d'execució és una mancança respecte del que l'API permet, però amb aquesta càrrega gràfica no és el que determina el rendiment observat. Adquiriria importància amb una escena que generés prou ordres com perquè l'enregistrament dominés la iteració, situació que aquest treball no arriba a plantejar.

6.5 Limitacions de l'avaluació

L'avaluació presentada té un abast limitat i convé enunciar-lo amb precisió, ja que condiciona quines conclusions se'n poden extreure.

Dos equips, dos controladors Les mesures provenen de dues màquines, cosa que permet contrastar arquitectures de memòria però no generalitzar. El repartiment de tipus de memòria, la mida de la finestra accessible des de l'amfitrió i el cost de les operacions de còpia són decisions d'implementació que poden variar entre controladors, entre versions del mateix controlador i entre models de la mateixa família.

Cost d'estratègia, no amplada de banda Com s'ha exposat, les dues branques comparades no executen la mateixa feina i les escriptures a la finestra de l'equip B no s'esperen. Les xifres no permeten afirmar res sobre el temps que les dades triguen a ser utilitzables pel dispositiu, que és la magnitud rellevant per a transferències grans.

Escena mínima La càrrega gràfica consisteix en un cub texturat i un quadrilàter d'interfície. No és representativa de cap càrrega real, i les conclusions sobre el repartiment de temps entre amfitrió i dispositiu només valen per a aquest règim.

Mesures per *frame* poc fiables Com s'ha documentat a la secció de metodologia, l'absència d'un període d'escalfament descartat i la durada curta de les execucions produeixen una variància entre execucions superior a les diferències que caldria distingir. Les conclusions d'aquest capítol es recolzen, per tant, únicament en els acumuladors de vida sencera, que no pateixen aquest problema.

Un experiment de quatre Dels quatre experiments plantejats només se n'ha completat un. Els tres restants depenen de mesures per *frame* i, per tant, de resoldre la limitació anterior.

Confusió entre treball i espera La secció que cronometra el renderitzat engloba tant l'enregistrament d'ordres com les esperes de sincronització, que són de naturalesa diferent. Mentre no se separin, qualsevol atribució del cost a una de les dues és una hipòtesi.

Cap d'aquestes limitacions afecta el resultat de l'experiment de transferència, que es recolza en acumuladors de vida sencera, en un control que es manté estable i en una diferència de dos ordres de magnitud. Sí que afecten, en canvi, l'abast de les afirmacions sobre rendiment general del motor, que en aquest capítol es limiten deliberadament al que les dades sostenen.

7 Conclusions

7.1 Conclusions del treball realitzat

El treball ha produït un motor de jocs funcional escrit en C, amb un renderitzador basat en Vulkan capaç de carregar geometria, textures i materials des de disc, il·luminar-los i dibuixar-los sobre dues plataformes. El balanç detallat dels objectius es recull al capítol 6; aquesta secció se centra en què ha establert el treball més enllà del producte.

7.1.1 L'API imposa l'arquitectura

La tesi que travessa la memòria és que la forma d'un motor de jocs modern no és majoritàriament una qüestió de gust, sinó que està determinada per la naturalesa de l'API sobre la qual es construeix. El capítol 2 l'enuncia a partir de les fonts: com que Vulkan trasllada a l'aplicació la gestió de riscos d'accés, la sincronització i l'assignació de memòria, qualsevol motor construït al damunt necessita un assignador, un model explícit de ritme de *frames* i un sistema de recursos propi.

El desenvolupament ho ha confirmat en la pràctica: són exactament els tres subsistemes als quals el capítol 5 dedica una secció pròpia, i l'Annex A en lliga cadascun amb la responsabilitat concreta que l'API delega i amb el fitxer que la implementa. Cap dels tres no respon a una preferència de disseny.

Que aquesta correspondència sigui un resultat i no una coincidència es pot argumentar de dues maneres, i convé fer-ho perquè és la tesi que sosté la memòria sencera.

La primera és de forma. Dues terceres parts dels punts on el motor crida l'API —110 de 161— serveixen per crear, consultar o destruir objectes, i només una tercera part per fer-los servir durant l'execució, tal com desglossa la secció sobre el cost de l'explicitud. Un motor sobre aquesta API no dedica la major part del seu codi gràfic a dibuixar, sinó a construir i mantenir l'estat que el model clàssic mantenia pel seu compte. Aquesta proporció no és una decisió del motor.

La segona és externa al treball. Si l'assignador de memòria de dispositiu fos una preferència de disseny, cada motor n'adoptaria una de diferent; el que s'observa és que existeix una biblioteca d'assignació de memòria mantinguda pel fabricant i adoptada de manera generalitzada [20], cosa que només té sentit si tots els motors necessiten resoldre el mateix problema de la mateixa manera. La seva existència és, doncs, evidència a favor de la tesi i no en contra: confirma que el subsistema és obligatori, i que l'única elecció disponible és escriure'l o adoptar-ne un de fet.

7.1.2 Fonts independents que convergeixen

Un segon resultat, menys previsible, és la freqüència amb què les dues fonts principals arriben a la mateixa solució per camins diferents.

El cas més clar és el patró d'inicialització dels subsistemes. La bibliografia d'arquitectura de motors recomana un ordre d'arrencada explícit perquè els mecanismes implícits del llenguatge no són fiables [1]; l'API imposa un dimensionament explícit perquè no reserva memòria en nom de l'aplicació [5]. Cap de les dues parla de l'altra, i el patró que el motor fa servir és la composició d'ambdues.

El mateix passa amb l'ordenació dels recursos de *shader* per freqüència d'actualització, descrita a §5.3: la bibliografia la recomana per raons de rendiment anteriors a Vulkan [1] i l'especificació per raons relatives a la invalidació de conjunts de descriptors [5]. Aquesta convergència suggereix que una part considerable de l'estructura d'un motor no és negociable.

7.1.3 El cost de l'explicitud i on surt a compte

Les dues seccions anteriors estableixen que l'API determina l'arquitectura. La pregunta que se'n deriva, i que és la més útil per a qui hagi de prendre la mateixa decisió, és una altra: què costa aquesta determinació i què s'hi guanya a canvi. El treball permet respondre-la amb dades pròpies en lloc d'amb expectatives.

7.1.3.1 Què costa

El cost és mesurable en codi. La Taula 12 en recull el repartiment, comptant les línies no buides dels fitxers font del motor i excloent-ne el codi de tercers.

Taula 12: Distribució del codi propi del motor per àrea, en línies no buides dels 84 fitxers font. No s'hi comptabilitzen les 7.016 línies del descodificador d'imatges de tercers que el motor incorpora.

Àrea	Línies	Proporció
Renderitzador, part específica de Vulkan	3.983	37,0 %
Sistemes bàsics	2.042	18,9 %
Matemàtiques	1.269	11,8 %
Sistemes especialitzats	1.177	10,9 %
Plataforma	1.078	10,0 %
Recursos	406	3,8 %
Contenidors	341	3,2 %
Renderitzador, part independent de l'API	267	2,5 %
Punt d'entrada	158	1,5 %
Assignadors	58	0,5 %
Total	10.779	100 %

Més d'un terç del codi del motor és, doncs, específic de Vulkan, i el renderitzador sencer s'acosta al quaranta per cent. Aquestes 3.983 línies contenen 161 crides a l'API, 22 funcions de creació d'objectes diferents i 42 estructures de descripció que cal omplir camp a camp abans de cada creació.

La xifra en codi, però, no és el cost complet. Tres dels onze subsistemes del motor — l'assignador, el model de ritme de *frames* i el sistema de recursos— existeixen perquè l'API no els proporciona, tal com argumenta la primera secció d'aquest capítol. El cost real inclou, doncs, també el temps d'entendre per què calen i què han de garantir, que el pressupost del capítol 3 situa en la partida més gran de les nou.

Convé subratllar la propietat que fa aquest cost incòmode: és fix. No es paga en proporció a l'ambició del motor, sinó abans de dibuixar el primer triangle. Presentar una imatge a la pantalla exigeix cadena d'intercanvi, passada de renderitzat, canonada gràfica, conjunts de descriptors, tanques i semàfors, tinguin el motor dos objectes o dos-cents mil.

El repartiment de les 161 crides a l'API ho quantifica. Setanta-una serveixen per crear objectes o consultar propietats del dispositiu i trenta-nou per destruir-los: cent deu de les cent seixanta-una, dues terceres parts, pertanyen a l'arrencada i al tancament. Només cinquanta-una s'executen durant el funcionament del motor, i d'aquestes la meitat són ordres d'enregistrament

al *command buffer*. Dit altrament, dos terços de la superfície d'API que el motor fa servir existeixen per arribar a poder dibuixar, no per dibuixar.

7.1.3.2 Què s'hi guanya, segons les fonts

El benefici que la bibliografia atribueix al model explícit és específic i està quantificat. El cost dominant del model clàssic és el treball que el controlador fa per cada ordre de dibuix, de manera que la millora apareix en escenes limitades per aquest treball, on es va mesurar en un factor d'entre cinc i quinze [2]. A aquest se n'hi suma un segon: l'enregistrament d'ordres es pot repartir entre fils, cosa que el model clàssic no permetia [5].

Cap dels dos no s'aplica a aquest motor. El renderitzador enregistra en un sol fil, tal com recull la secció de limitacions, i l'escena consta de dues crides de dibuix per iteració: una geometria de món i un quadrilàter d'interfície. El criteri que les mateixes fonts proposen situa aquest motor, per tant, lluny del règim on el model clàssic era el límit.

Les mesures ho confirmen en lloc de deixar-ho com a raonament. El temps que el dispositiu dedica a executar una iteració no supera el setze per cent del temps total d'aquesta iteració i se situa habitualment al voltant del cinc per cent (§6.4). El motor no està esperant la GPU, de manera que el benefici que la bibliografia anuncia no tenia marge on manifestar-se.

7.1.3.3 Què s'hi ha guanyat realment

Hi ha, en canvi, un eix on l'explicitud ha donat un resultat gran, i no és el que els materials consultats posen al davant. Poder consultar els tipus de memòria que el dispositiu exposa i triar l'estratègia de transferència en conseqüència redueix el temps de càrrega de geometria en un factor de cent trenta-sis a l'equip de memòria unificada i de dos-cents vuitanta-dos al de GPU dedicada (§6.3).

Aquesta millora té dues propietats que la fan rellevant per a la pregunta d'aquesta secció. La primera és que no depèn de l'ambició del motor: apareix amb dues geometries igual que n'apareixeria amb dues mil, perquè el que mesura és el camí de transferència i no la càrrega gràfica. La segona és que cap API que amagui la ubicació de la memòria no la pot oferir, perquè la decisió que la produeix —escriure directament en memòria local al dispositiu quan n'hi ha de visible des de l'amfitrió— no existeix com a opció si el controlador la pren pel seu compte.

7.1.3.4 El criteri que se'n desprèn

La resposta a la pregunta inicial és, doncs, asimètrica, i aquesta asimetria és el resultat més útil del treball per a qui hagi de decidir.

El cost de l'explicitud és fix i es paga per endavant. El benefici que la bibliografia anuncia —reducció del treball del controlador i enregistrament paral·lel— és proporcional a l'ambició del motor i, per sota d'un determinat volum d'ordres de dibuix, senzillament no arriba. Un motor petit paga, per tant, la totalitat del cost a canvi de cap de les dues millores amb què l'API es justifica habitualment.

El benefici del control sobre la ubicació de les dades, en canvi, no és proporcional a res: apareix complet des del primer actiu que es carrega. És l'únic dels tres que un motor de l'abast d'aquest pot cobrar.

D'això se'n segueix un criteri concret. La pregunta pertinent abans d'adoptar una API explícita no és quantes ordres de dibuix tindrà l'escena, que és la que la bibliografia suggereix, sinó si el projecte necessita decidir on resideixen les seves dades. Per a un motor petit, l'argument de rendiment per *frame* no se sosté; l'argument de control sobre la memòria sí.

Queda una raó que no és tècnica i que en aquest treball és decisiva. Quan l'objectiu és entendre com funciona un motor, el cost de l'explicitud no és un preu sinó el contingut: són precisament les responsabilitats que l'API delega les que es volien estudiar, i una API que les amagués hauria fet el treball impossible. Això justifica l'elecció per a aquest projecte sense estendre-la a projectes amb un altre objectiu.

7.1.4 L'explicitud desplaça la portabilitat a l'aplicació

El treball ha produït cinc casos del mateix error, i la seva acumulació permet enunciar una conseqüència del model explícit que les fonts no destaquen tant com el rendiment.

Quatre d'ells —selecció de dispositiu, extensió de la superfície, dimensionament de recursos per imatge i límit de filtratge anisotròpic— van impedir que el motor s'executés sobre el maquinari d'aquest treball, i es detallen a §5.3. El cinquè, la còpia intermèdia innecessària sobre memòria unificada, no impedia res: només consumia temps, i per això només es podia detectar mesurant.

Tots cinc comparteixen la mateixa causa: un valor escrit al codi en lloc de consultat al dispositiu. Enunciat així sembla una qüestió de disciplina, i no ho és. En el model clàssic, aquests cinc valors els decidia el controlador, que els ajustava a cada dispositiu sense que l'aplicació ho sabés; aquella capa d'indirecció era, alhora, el cost que el model explícit elimina i el mecanisme que feia portable el codi de l'aplicació. Suprimir-la trasllada la responsabilitat sencera, no només la part que interessa.

La conseqüència pràctica és que el preu del control no es cobra en temps de desenvolupament de l'escena sinó en enginyeria de portabilitat, i es cobra sobre maquinari que qui escriu el codi no té. Els quatre primers casos eren correctes a la màquina on es van escriure i van fallar a la primera màquina diferent. La documentació de Mantle ja ho advertia el 2015 [4] i l'especificació de Vulkan ho manté [5]; haver-ho trobat cinc vegades de manera independent li dona un pes que llegir-ho no li donava.

7.1.5 Una implementació de referència transmet també el seu entorn

El treball s'ha construït sobre una implementació de referència, pràctica que el capítol 2 documenta com a habitual en el desenvolupament de motors [1]. Aquesta decisió, declarada al capítol 3, ha permès abastar en el temps disponible un conjunt de subsistemes que difícilment s'hauria assolit partint de zero. Que aquesta manera de treballar sigui legítima no és, però, cap resultat: el capítol 3 ja la justifica i ningú no la discuteix.

El resultat és un altre, i és menys evident. Els quatre supòsits sobre el dispositiu de la secció anterior no eren errors de la implementació de referència: eren certs a la màquina del seu autor. El codi els expressava amb la mateixa forma amb què expressa les decisions de disseny, de manera que res no els distingia d'aquestes fins que van fallar. Una implementació de referència transmet, doncs, dues coses alhora: una arquitectura i un entorn d'execució, i només la primera és explícita.

D'aquí se'n deriva el criteri metodològic que el treball ha acabat aplicant, i que és més precís que el que el capítol 3 enuncitava en començar: el contrast amb les fonts primàries no serveix per validar l'arquitectura que la referència proposa, que és sòlida, sinó per separar-ne el que l'API garanteix del que simplement era cert en un lloc concret. Els dos casos en què l'estudi de les fonts va modificar una conclusió ho il·lustren. El primer és l'argument sobre l'elecció del llenguatge de §4.2, on la font que es pretenia citar afirma el contrari del que se li volia atribuir. El segon és

l'experiment de §6.3, on una pràctica que tots els materials consultats presenten com a correcta resulta prescindible sobre el maquinari emprat. Cap dels dos no s'hauria produït reproduint codi.

7.1.6 Sobre construir l'instrument

El capítol 1 planteja, com a motivació, una consideració sobre el mitjà: que al videojoc, a diferència de gairebé qualsevol altra disciplina creativa, qui concep l'obra sovint està capacitat per construir també l'instrument. Convé tancar-la amb el que el treball permet dir-ne, que és menys del que la motivació esperava i més concret.

Les prop de quatre-centes hores que el capítol 3 comptabilitza han produït un motor capaç de carregar geometria, textures i materials des de disc, il·luminar-los amb una llum direccional i dibuixar-los sobre dues plataformes. La distància respecte de qualsevol motor comercial no és de grau sinó d'ordre de magnitud, i la secció de limitacions d'aquest capítol l'enumera sense estalviar-se'n cap.

La conclusió honesta, per tant, no és que construir l'instrument sigui una alternativa a fer-lo servir. És que són activitats amb objectius diferents, i que el valor de la primera no s'ha de mesurar amb la vara de la segona. El que aquest treball ha produït que un motor comercial no hauria produït no és el programa, sinó saber per què el bucle principal té les fases que té, on és la memòria de cada actiu i què passa quan el maquinari no és el previst. Vist així, la separació entre l'instrument i l'obra que el capítol 1 lamenta no és un problema a resoldre sinó una divisió del treball raonable, amb l'excepció que qui la travessa hi guanya un coneixement del mitjà que no s'obté de cap altra manera.

7.2 Punts forts i punts febles

7.2.1 Punts forts

Decisions documentades amb el seu cost Cada decisió estructural del capítol 4 enuncia les alternatives considerades, el criteri aplicat i què s'hi va perdre. Aquesta última part hi consta sempre, inclosa la fuga reconeguda del renderitzador i les tres limitacions del model de memòria.

Adaptació a maquinari no previst Els quatre supòsits corregits a §5.3 són treball de diagnòstic independent, no reproduït de cap material, i van exigir distingir què garanteix l'API de què deixa a criteri de la implementació.

Instrumentació pròpia Tot el capítol 6 es recolza en una capa de mesura escrita per al projecte, incloent-hi la mesura de temps de dispositiu, sense recórrer a cap eina externa.

Un resultat empíric amb valor propi L'experiment de transferència contradiu una pràctica establerta sobre una classe de maquinari molt estesa, amb un control que permet atribuir-ne el resultat.

Delimitació honesta de l'abast Tant el capítol 6 com aquest distingeixen quines afirmacions sostenen les dades i quines no, en lloc d'agrupar totes les limitacions indistintament.

7.2.2 Punts febles

Proporció del treball derivat La major part dels sistemes segueixen l'estructura de la implementació de referència. El treball propi hi consisteix a haver-los comprès, integrats, depurats i documentats, no a haver-los concebut, i el capítol 3 ho delimita explícitament.

Avaluació incompleta S'ha completat un dels quatre experiments enumerats a §6.3, si bé sobre els dos equips previstos. Els tres restants depenen de mesures per *frame* i, per tant, del defecte de l'instrument que s'assenyala tot seguit.

Instrument de mesura millorable L'absència d'un període d'escalfament descartat inutilitza les mesures per *frame* per a distincions fines, i és precisament el que bloqueja els tres experiments restants.

Documentació tècnica desactualitzada El corpus de documents del repositori, que el capítol 3 presenta com a part del mètode, no recull les ampliacions més recents i descriu mòduls que la reorganització de passades va eliminar.

Cobertura de proves reduïda Les proves unitàries cobreixen únicament l'assignador lineal i la taula de dispersió. La resta del motor es verifica per execució, cosa que detecta regressions visibles però no silencioses.

7.3 Limitacions

Més enllà de la qualitat del treball realitzat, el motor té limitacions estructurals que en delimiten l'abast. Es recullen aquí les que es consideren rellevants, totes documentades al lloc corresponent.

Un sol fil d'execució L'enregistrament d'ordres es fa íntegrament en un fil. L'API està dissenyada per permetre'n l'enregistrament paral·lel i especifica quins objectes requereixen sincronització externa per fer-ho possible [5]; el motor no ho aprofita. És la limitació més rellevant respecte del que l'API ofereix, si bé les mesures de §6.4 indiquen que amb la càrrega gràfica actual no és el que determina el rendiment observat.

Absència de representació d'escena El paquet de renderitzat es construeix amb geometries fixes al codi de l'aplicació. No hi ha jerarquia d'objectes, ni descart per visibilitat, ni ordenació de la feina de dibuix.

Gestió de memòria de dispositiu elemental Els búfers de geometria es reserven en blocs grans i fixos sense subassignador, de manera que el motor no pot alliberar ni reutilitzar regions de manera selectiva.

Assignadors sense alineació Cap dels dos assignadors de l'amfitrió alinea les reserves, i l'assignador lineal no comprova el punter nul que retorna en exhaurir-se.

Unificació parcial de recursos El sistema de recursos unifica la localització i la càrrega, però tres subsistemes mantenen cadascun la seva taula de referències i la seva política d'alliberament.

Il·luminació mínima Una sola llum direccional, sense llums puntuals, sense ombres i amb l'exponent especular fixat al *shader* en lloc de formar part del material. La transformació de normals empra la submatriu del model, cosa que deixaria de ser correcta amb escalat no uniforme.

7.4 Línies de continuació

Les línies següents s'enuncien per ordre de prioritat, entenent com a tal la relació entre el que aporten i el que costen.

1. **Completar l'avaluació.** Executar l'experiment de transferència sobre l'equip amb GPU dedicada. És la meitat que converteix l'avaluació en una comparació entre arquitectures, i no requereix cap codi nou: el motor ja selecciona l'estratègia consultant el dispositiu.

2. **Sanejar l'instrument de mesura.** Descartar un període d'escalfament, allargar les execucions i separar l'enregistrament d'ordres de les esperes de sincronització. Desbloqueja els tres experiments pendents i elimina la confusió entre treball i espera assenyalada a §6.4.
3. **Sistema de tasques concurrents.** És el canvi estructural de més abast disponible i el que més aprofitaria el disseny de l'API. La bibliografia en descriu l'arquitectura [1], [24] i l'especificació precisa quins objectes requereixen sincronització externa [5].
4. **Cua de renderitzat i representació d'escena.** Substituir el paquet fix per una llista de parells de malla i material, ordenable per minimitzar canvis d'estat [1], [25]. És el requisit previ de qualsevol escena no trivial.
5. **Subassignador de memòria de dispositiu.** Repartir les reserves grans que el motor ja fa, tal com recomana la documentació de referència [4].
6. **Assignador de *frame*.** Afegir marcadors i retorn a l'assignador lineal per obtenir l'assignador de *frame* que la bibliografia descriu [1], i reiniciar-lo a cada iteració.
7. **Completar la unificació de recursos.** Traslladar el recompte de referències i la identificació per nom al sistema de recursos, de manera que els sistemes especialitzats només aportin la interpretació de cada tipus.
8. **Alineació als assignadors i comprovació del punter nul.** Són canvis petits amb efecte directe sobre la correcció.

Referències

- [1] J. Gregory, *Game Engine Architecture*, 3r ed. CRC Press, 2018.
- [2] C. Everitt, «OpenGL Efficiency: AZDO», dins *Game Developers Conference*, San Francisco, CA, març 2014. [En línia]. Disponible a: <https://www.khronos.org/assets/uploads/developers/library/2014-gdc/Khronos-OpenGL-Efficiency-GDC-Mar14.pdf>
- [3] T. Olson, «Vulkan 101». 2016.
- [4] G. Riguer, «Mantle Programming Guide and API Reference». març de 2015.
- [5] Khronos Vulkan Working Group, «Vulkan 1.4.361 — A Specification (with all registered extensions)». 2026. [En línia]. Disponible a: <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html>
- [6] Unity Technologies, «Unity Documentation». [En línia]. Disponible a: <https://docs.unity3d.com/>
- [7] Epic Games, «Unreal Engine Documentation». [En línia]. Disponible a: <https://dev.epicgames.com/documentation/unreal-engine>
- [8] Godot Engine contributors, «Godot Engine Documentation». [En línia]. Disponible a: <https://docs.godotengine.org/>
- [9] T. Vroman, «Kohi Game Engine / Shadows of Illumina (Vulkan Game Engine and Game Written in C)», *YouTube*, 2021, [En línia]. Disponible a: <https://www.youtube.com/playlist?list=PLv8Ddw9K0JPg1BEO-RS-0MYs423cvLVtj>
- [10] T. Vroman, «kohi — A game engine made as part of the Kohi Game Engine series, written in C and Vulkan». [En línia]. Disponible a: <https://github.com/travisvroman/kohi>
- [11] C. Muratori, «Handmade Hero». [En línia]. Disponible a: <https://handmadehero.org/>
- [12] B. Karadžić, «bgfx — Cross-platform rendering library». [En línia]. Disponible a: <https://github.com/bkaradzic/bgfx>
- [13] S. Willems, «Vulkan C++ examples and demos». [En línia]. Disponible a: <https://github.com/SaschaWillems/Vulkan>
- [14] The Khronos Group, «MoltenVK — A Vulkan Portability implementation over Apple's Metal graphics framework». [En línia]. Disponible a: <https://github.com/KhronosGroup/MoltenVK>
- [15] B. Stroustrup, *The C++ Programming Language*, 4t ed. Addison-Wesley, 2013.
- [16] J. Gustedt, *Modern C*, 2n ed. Manning Publications, 2019.
- [17] R. Fabian, *Data-Oriented Design*. 2018. [En línia]. Disponible a: <https://www.dataorienteddesign.com/dodbook.pdf>
- [18] A. Weissflog, «Handles are the better pointers», *The Brain Dump*, juny 2018, [En línia]. Disponible a: <https://floohh.github.io/2018/06/17/handles-vs-pointers.html>
- [19] B. Hall, «Memory Allocation Strategies», *gingerBill*, feb. 2019, [En línia]. Disponible a: <https://www.gingerbill.org/series/memory-allocation-strategies/>

- [20] A. Sawicki, «Vulkan Memory Allocator — Documentation». [En línia]. Disponible a: <https://gpuopen-librariesandsdks.github.io/VulkanMemoryAllocator/html/>
- [21] U. Drepper, «What Every Programmer Should Know About Memory», *Red Hat, Inc.*, 2007, [En línia]. Disponible a: <https://people.redhat.com/drepper/cpumemory.pdf>
- [22] H.-K. Arntzen, «Yet another blog explaining Vulkan synchronization», *Maister's Graphics Adventures*, ago. 2019, [En línia]. Disponible a: <https://themaister.net/blog/2019/08/14/yet-another-blog-explaining-vulkan-synchronization/>
- [23] G. Fiedler, «Fix Your Timestep!». [En línia]. Disponible a: https://gafferongames.com/post/fix_your_timestep/
- [24] C. Gyrlling, «Parallelizing the Naughty Dog Engine Using Fibers», dins *Game Developers Conference*, San Francisco, CA, 2015.
- [25] S. Reinalter, «Stateless, layered, multi-threaded rendering», *Molecular Musings*, nov. 2014, [En línia]. Disponible a: <https://blog.molecular-matters.com/2014/11/06/stateless-layered-multi-threaded-rendering-part-1/>

Annexos

Annex A. Correspondència entre decisions i fonts

Aquest annex recull, de manera compacta i verificable, la correspondència que l'objectiu de documentació enunciat al capítol 1 promet: cada mecanisme estructural del motor, la secció de la memòria que el justifica, la font primària que el fonamenta i el fitxer del codi que l'implementa. Serveix per comprovar qualsevol afirmació del capítol 4 o del 5 sense haver de recórrer el text sencer.

La Taula 13 en recull el contingut. Les rutes són relatives a `engine/src/`, les referències numèriques corresponen a la bibliografia i les seccions precedides d'un número entre claudàtors són les de l'obra citada, no les d'aquesta memòria.

Taula 13: Correspondència entre els mecanismes del motor, la seva justificació a la memòria, la font que els fonamenta i el codi que els implementa.

Mecanisme	Memòria	Font	Codi
<i>Arquitectura i cicle de vida</i>			
Motor com a <i>framework</i> : el motor posseeix el bucle	§4.3	[1] §8.3.2	<code>entry.h</code> , <code>game_types.h</code>
Ordre d'arrencada explícit dels subsistemes	§4.4, §5.1	[1] §6.1	<code>core/application.c</code>
Dimensionament en dues crides	§4.4, §5.1	[5] §5.1	tots els <code>*_initialize</code>
Capa d'abstracció de plataforma	§4.1, §5.1	[1] §1.6.5	<code>platform/</code>
Superfície de finestra mitjançant extensió	§5.1	[5] §40.1	<code>vulkan/vulkan_platform.h</code>
<i>Gestió de memòria</i>			
Reserva etiquetada i estadístiques d'ocupació	§5.2	[1] §10.9	<code>core/hmemory.c</code>
Assignador lineal sobre bloc fix	§4.4, §5.2	[1] §6.2.1.2, [19]	<code>memory/linear_allocator.c</code>
Alineació de reserves (manca documentada)	§5.2	[1] §3.3.7, [21]	<code>core/hmemory.c</code>
Consulta de munts i tipus de memòria del dispositiu	§5.2, §6.3	[5] §3.2, [4]	<code>vulkan/vulkan_backend.c</code>
Transferència mitjançant búfer intermedi	§5.2, §6.3	[4], [20]	<code>vulkan/vulkan_backend.c</code>
<i>Renderitzador</i>			
Separació entre part independent de l'API i implementació	§4.5	[1] §1.6.5	<code>renderer/renderer_types.inl</code>
Enviament asíncron: sense ordenació implícita	§2.2, §5.3	[5] §3.2	<code>vulkan/vulkan_backend.c</code>
Tanques: dependència de cua cap a l'amfitrió	§5.3	[5] §7.3, [22]	<code>vulkan/vulkan_backend.c</code>

Mecanisme	Memòria	Font	Codi
Semàfors: dependència entre operacions de cua	§5.3	[5] §7.4, [22]	vulkan/vulkan_backend.c
Conjunts de descriptors ordenats per freqüència	§4.6, §5.3	[5] §17.2	vulkan/shaders/vulkan_material_shader.c
Constants d'inserció per a la dada de màxima freqüència	§5.3	[5] §17.10	vulkan/shaders/vulkan_material_shader.c
<i>Shaders</i> compilats a SPIR-V abans de l'execució	§2.4, §3.4	[5] §9.2	post-build.sh, post-build.bat
Capes de validació només en compilacions de depuració	§2.2, §5.1	[3], [5] §59	vulkan/vulkan_backend.c
<i>Recursos</i>			
Recompte de referències com a política de cicle de vida	§4.6, §5.4	[1] §7.2.4	systems/texture_system.c i anàlegs
Generacions com a detecció d'obsolescència	§4.6, §5.4	[1] §16.5, [18]	resources/resource_types.h
Capa unificada de localització i càrrega d'actius	§5.4	[1] §7.1	systems/resource_system.c
Actiu dirigit per dades en fitxer de text	§5.4	[1] §15.3	resources/loaders/material_loader.c
<i>Instrumentació</i>			
Perfilatge integrat per fases	§5.2, §6.1	[1] §10.8	core/metrics.c
Mesura de temps de dispositiu amb marques de temps	§5.2, §6.1	[5] §23	vulkan/vulkan_backend.c

Cal fer una precisió sobre l'abast d'aquesta taula. Recull els mecanismes que tenen una font primària identificable, que són els que el capítol 4 documenta com a decisions i els que el capítol 5 descriu en profunditat. No hi consten els subsistemes la implementació dels quals segueix la referència sense que cap decisió estructural pròpia n'hagi requerit fonamentació independent; aquests es recullen a l'inventari de §5.5 i la seva procedència es delimita al capítol 3.

Annex B. Compilació i execució

Aquest annex recull el que cal per compilar i executar el motor sobre les dues plataformes en què s'ha desenvolupat. La informació es manté també al fitxer README.md del repositori.

El codi font complet, la documentació tècnica dels subsistemes i l'historial de *commits* que el capítol 3 pren com a registre cronològic del desenvolupament són accessibles a:

<https://github.com/OleguerAlmuni/hefest>

Requisits comuns

Clang El motor i l'aplicació de proves es compilen i s'enllacen exclusivament amb aquest compilador. Les versions emprades consten a la Taula 5.

GNU Make El procés de construcció es governa amb fitxers de construcció separats per component i per sistema operatiu.

Eines de Vulkan Cal disposar del compilador de *shaders glslc*, que tradueix el GLSL a SPIR-V. La via per obtenir-lo difereix entre plataformes i es detalla més avall.

Linux

Calen, a més, els paquets de desenvolupament de X11 i XCB: `libx11-dev`, `libxcb1-dev`, `libx11-xcb-dev` i `libxkbcommon-x11-dev`, o els seus equivalents segons la distribució. Les eines de Vulkan poden provenir dels paquets de la distribució, cas en què no cal definir cap variable d'entorn.

El Codi 6 recull la seqüència, des de l'arrel del repositori.

```
./build-all.sh    # compila motor, aplicació de proves i proves unitàries
./post-build.sh   # compila els shaders a SPIR-V i copia assets/ a bin/
./run.sh          # executa des de bin/, on es troba libengine.so
```

Codi 6: Construcció i execució sobre Linux.

L'aplicació s'ha d'executar des del directori `bin/`, ja que l'enllaçat cerca la biblioteca compartida al directori de treball. El script `run.sh` ja ho té en compte.

Windows

Cal el SDK de Vulkan de LunarG i que la variable d'entorn `VULKAN_SDK` estigui definida, atès que tant els fitxers de construcció com el pas posterior la fan servir per localitzar les eines. El Codi 7 en recull la seqüència.

```
build-all.bat
post-build.bat
bin\testbed.exe
```

Codi 7: Construcció i execució sobre Windows.

Notes d'ús

El pas posterior a la construcció no s'executa automàticament, de manera que qualsevol modificació dels *shaders* sota `assets/shaders/` exigeix tornar-lo a executar perquè es recompilin.

Les proves unitàries es generen com a executable independent a `bin/tests`.

L'estratègia de transferència de dades cap a la memòria del dispositiu, emprada a l'experiment del capítol 6, se selecciona amb la variable d'entorn `HEFEST_DIRECT_UPLOAD`: amb valor 1 el motor intenta l'escriptura directa i, si el dispositiu no exposa el tipus de memòria necessari, torna a la via del búfer intermedi i ho registra. Sense definir, o amb valor 0, empra sempre el búfer intermedi.

Annex C. Registres de les mesures

Les execucions que sostenen les xifres del capítol 6 es conserven senceres al repositori, sota `docs/tfg/mesures/`, amb el nom de l'equip i de la configuració al fitxer. No s'han retallat ni editat: contenen també la sortida del carregador de Vulkan i l'enumeració de capes, de manera que se'n pot verificar l'entorn d'execució a més dels resultats.

Cada registre conté tres línies rellevants per a l'experiment:

Estratègia efectiva La línia que comença per `Upload strategy`: indica quina via va emprar realment el motor, que no ha de coincidir necessàriament amb la sol·licitada: si el dispositiu no exposa un tipus de memòria alhora local i visible des de l'amfitrió, el motor torna a la via del búfer intermedi i ho fa constar.

Temps de càrrega La línia que comença per uploads recull el nombre d'operacions i el temps acumulat, separats entre búfers de geometria i imatges. És la darrera línia de cada informe i la font de la Taula 11.

Informe de temps per *frame* El bloc encapçalat per `Frame timing over` recull la finestra de mostres. Se n'emet un per segon; el rellevant és l'últim de cada execució.

Els registres de l'equip B provenen del fitxer que el motor escriu a `bin/console.log` i no de la redirecció de la sortida estàndard, ja que sobre Windows la funció d'escriptura per consola de la capa de plataforma no funciona quan la sortida apunta a un fitxer. És una limitació de la implementació d'aquella capa que no afecta les mesures però sí la manera de capturar-les, i que el fitxer de notes del mateix directori documenta.